

Занятие 1. Справочник и ссылки

Расширение к учебному материалу [lesson-01.md](#): каждый блок занятия разобран подробнее, со ссылками на первоисточники. Все ссылки проверены открытием страницы **12 августа 2026 года** — для инструментов, которые меняются раз в недели, дата важнее самой ссылки.

Материал собран под запись первого занятия и предполагает, что вы её посмотрели: здесь нет пересказа демонстрации, только то, что за ней стоит.

Если агентами вы ещё не пользовались — начните не отсюда, а с [getting-started](#). Там первые шаги простым языком: какой интерфейс выбрать (терминал не обязателен), как запустить и как поручить агенту настройку проекта. Этот справочник пригодится следующим шагом, когда появятся конкретные вопросы.

Как этим пользоваться

Разделы независимы — читайте тот, который нужен прямо сейчас. Если проходите последовательно, рабочий порядок такой:

1. **Скиллы и Разрешения** — то, что настраивается в первый день и работает дальше всегда.
2. **MCP** — когда агенту не хватает доступа к инструменту.
3. **Хуки** — когда правило нужно не попросить, а обеспечить.
4. **superpowers / SDD** — когда задача больше одной сессии.
5. **Контекст и экономика** — когда упёрлись в лимиты или качество поплыло на длинной сессии.

Раздел **Репозитории** — навигация по первоисточникам, к нему возвращаются по мере надобности.

Границы проверки

Честный список мест, где источники расходятся или подтверждения нет. В самих разделах эти оговорки повторены по месту.

ЧТО	В ЧЁМ ПРОБЛЕМА
Звёзды <code>obra/superpowers</code> (~271k)	Аномально много для репозитория младше года. Цифру одинаково отдают REST API и страница репозитория, независимых подтверждений нет
Слэш-команды <code>superpowers</code>	Витрина на <code>claude.com</code> упоминает <code>/brainstorming</code> и <code>/execute-plan</code> , но каталога <code>commands/</code> в <code>v6.2.0</code> нет — вызывать нужно через <code>/superpowers:<skill></code>
Установка Claude Code	README репозитория и раздел <code>setup</code> документации расходятся: <code>npm</code> помечен то как <code>deprecated</code> , то как <code>advanced</code> -опция; требования к <code>Node.js</code> тоже разные (18+ против 22+)
Лицензия <code>anthropics/skills</code>	GitHub API отдаёт <code>null</code> , README заявляет Apache 2.0 плюс <code>source-available</code> для документных скиллов
Статус AAIF в Linux Foundation	Подтверждается только формулировкой самого <code>agents.md</code> ; структура на стороне LF не описана
Токенные квоты тарифов	Anthropic публикует только множители <code>1x / 5x / 20x</code> , точных лимитов в токенах нет. Страница <code>pricing</code> показывает <code>Max</code> «от \$100», справочный центр разделяет <code>Max 5x</code> и <code>Max 20x</code>
Поле с текстом плана в <code>ExitPlanMode</code>	Имя поля в <code>tool_input</code> не задокументировано — в примере хука JSON дампится целиком
Компиляции инцидентов на сторонних сайтах	В текст вошли только случаи с первоисточником: GitHub Security Advisories, Wiz, The Register, инженерный блог Anthropic

Содержание

1. Репозитории и официальные ресурсы — откуда брать первоисточники
2. Скиллы — компетенция агента: устройство, установка, свои скиллы
3. `superpowers` и Spec-Driven Development — флоу «спека → план → код → проверка»
4. MCP — подключение внешних инструментов и его цена
5. Хуки — обеспечить кодом, а не попросить словами
6. Контекст, модели и экономика — окно, компакт, усилие, тарифы
7. Разрешения и безопасность — режимы, песочницы, чек-лист

1. Репозитории и официальные ресурсы

откуда брать первоисточники

На занятии ссылки мелькали быстро. Здесь они собраны с проверкой: каждый URL открыт 12 августа 2026 года, для репозиторий зафиксированы звёзды и дата последней активности. Звёзды и версии устаревают за недели, поэтому ниже везде указано, **как проверить актуальное состояние самому**, а не только «как было». Там, где источники противоречат друг другу, это сказано прямо.

Claude Code

Документация живёт на code.claude.com/docs — не на docs.anthropic.com, куда до сих пор ведут многие статьи. Отдельно стоит запомнить code.claude.com/docs/llms.txt: это машиночитаемый индекс всех страниц документации, его удобно скармливать самому агенту.

Главная ловушка — репозиторий [anthropics/claude-code](https://github.com/anthropics/claude-code) (141.2k звёзд, последняя активность 11.08.2026). Это **не исходники CLI**. В корне лежат только `README.md`, `CHANGELOG.md`, `SECURITY.md`, `LICENSE.md`, каталоги `examples/`, `plugins/`, `scripts/` и служебные `.github/`, `.claude/`, `.devcontainer/`. Каталога `src/` там нет. Репозиторий работает как issue tracker, публикатор changelog и хранилище официальных плагинов. Страница code.claude.com/docs/en/changelog генерируется из `CHANGELOG.md` этого репозитория, так что обе ссылки равнозначны.

Версия. На занятии была 2.1.227 (вышла 10.08.2026), а уже 11.08 приехала 2.1.228 — на следующий день после записи. Это нормальный ритм: 2.1.224, 2.1.225 и 2.1.226 вышли за четыре дня до того. Не забывайте номер, запоминайте команды:

```
claude --version    # печатает, например: 2.1.228 (Claude Code)
claude doctor       # read-only диагностика установки и settings.json, без запуска сессии
claude update       # применить обновление немедленно
```

Установка. Рекомендованный способ — нативный инсталлятор, он единственный обновляется в фоне сам:

```
# macOS / Linux / WSL — можно указать канал или точную версию
curl -fsSL https://claude.ai/install.sh | bash
curl -fsSL https://claude.ai/install.sh | bash -s stable

# Homebrew: два разных cask
brew install --cask claude-code          # канал stable, отстаёт примерно на неделю
brew install --cask claude-code@latest   # канал latest

# Windows
irm https://claude.ai/install.ps1 | iex
winget install Anthropic.ClaudeCode
```

Homebrew, WinGet, apt/dnf/apk сами не обновляются. Канал переключается настройкой `autoUpdatesChannel` ("latest" / "stable") в `settings.json`; соседний `minimumVersion` не даёт откатиться вниз.

Противоречие в источниках. README репозитория помечает npm-установку как deprecated и вешает бейдж «Node.js 18+». Страница `docs/en/setup` npm deprecated не называет — описывает как advanced-опцию и указывает, что начиная с v2.1.198 пакет требует **Node.js 22+**. Ориентируйтесь на документацию: npm ставит тот же нативный бинарник, что и инсталлятор, и Node.js в рантайме не использует.

Codex CLI от OpenAI

Репозиторий — [openai/codex](https://github.com/openai/codex), 105.5k звёзд, лицензия Apache-2.0, активность ежедневная (последний push 12.08.2026). Документация **переехала**: developers.openai.com/codex отдаёт 308 и ведёт на learn.chatgpt.com/docs. Старые ссылки работают через редирект, но в закладки лучше класть новый адрес.

```
curl -fsSL https://chatgpt.com/codex/install.sh | sh # macOS / Linux
npm install -g @openai/codex
brew install --cask codex
```

Нумерация версий сбивает с толку. Релизы на GitHub тегируются как `rust-vX.Y.Z` — последний стабильный `rust-v0.147.0` (07.08.2026), параллельно почти ежедневно выходят альфы `rust-v0.148.0-alpha.*`. При этом `codex --version` печатает строку вида `codex-cli 0.131.0`. Сверять тег релиза с выводом команды напрямую нельзя.

codex exec — неинтерактивный режим для скриптов и CI. Синтаксис:

```
codex exec [OPTIONS] [PROMPT]
codex exec [OPTIONS] <COMMAND> [ARGS] # подкоманды: resume, review
```

Промпт можно передать аргументом, через `-`, или подать на `stdin` — тогда он придет отдельным блоком `<stdin>`. Флаги (проверено выводом `codex exec --help`):

ФЛАГ	ЧТО ДЕЛАЕТ
<code>--skip-git-repo-check</code>	Allow running Codex outside a Git repository
<code>-C, --cd <DIR></code>	рабочий корень агента
<code>--add-dir <DIR></code>	дополнительные каталоги, доступные на запись
<code>-s, --sandbox <MODE></code>	<code>read-only</code> <code>workspace-write</code> <code>danger-full-access</code>
<code>--json</code>	события в stdout как JSONL
<code>-o, --output-last-message <FILE></code>	последнее сообщение агента в файл
<code>--output-schema <FILE></code>	JSON Schema для формы финального ответа
<code>--ephemeral</code>	не писать файлы сессии на диск
<code>--ignore-user-config</code>	не читать <code>\$CODEX_HOME/config.toml</code>
<code>-c, --config key=value</code>	точечный override конфига (значение парсится как TOML)

По умолчанию Codex отказывается работать вне git-репозитория: репозиторий — это страховка, из которой можно откатить всё, что агент натворил. `--skip-git-repo-check` снимает именно эту проверку и ничего больше. Осмысленные сценарии — генерация документа в `/tmp`, разбор логов, работа в каталоге, который под git класть незначает:

```
codex exec --skip-git-repo-check -C /tmp/notes --sandbox read-only \
"прочитай app.log и выпиши топ-10 повторяющихся ошибок" -o summary.md
```

Конвенция AGENTS.md

Сайт спецификации — [agents.md](#), репозиторий — [agentsmd/agents.md](#) (23.6k звёзд; сам сайт правился 12.03.2026, файл `AGENTS.md` внутри — 19.08.2025). Формат стюардит **Agentic AI Foundation** ([aaif.io](#)) — там же живут MCP, goose и agentgateway. Формулировку «under the Linux Foundation» даёт сам сайт [agents.md](#); страница проектов AAIF показывает брендинг Linux Foundation, но юридический статус явно не описывает. Заявленное распространение — более 60 тысяч открытых проектов.

Спецификация намеренно бедная: это обычный Markdown без обязательных полей и разделов. Единственное правило про структуру — вложенность: «Agents automatically read the nearest file in the directory tree», то есть ближайший к файлу `AGENTS.md` побеждает, и каждый подпроект монорепо может нести свой.

Claude Code `AGENTS.md` не читает. Документация формулирует это прямо: «Claude Code reads `CLAUDE.md`, not `AGENTS.md`». Мостов три:

```
# 1. импорт — работает везде, включая Windows
printf '@AGENTS.md\n' > CLAUDE.md
# 2. симлинк — если Claude-специфичных инструкций не будет
ln -s AGENTS.md CLAUDE.md
# 3. одnorазовое копирование конфигурации другого агента (нужен Claude Code v2.1.213+)
#   /import — прямо в сессии
```

	CLAUDE.MD	AGENTS.MD
Кто читает	только Claude Code	20+ агентов: Codex, Cursor, Copilot, Gemini CLI, Jules, Aider, Zed, Windsurf, Devin
Стандарт	продуктовая документация Anthropic	открытая спецификация под AAIF / Linux Foundation
Уровни	managed policy → <code>~/.claude/CLAUDE.md</code> → <code>./CLAUDE.md</code> или <code>./.claude/CLAUDE.md</code> → <code>./CLAUDE.local.md</code>	ближайший файл в дереве каталогов
Импорты	<code>@path/to/file</code> , до 4 уровней вложенности	нет
Рекомендуемый размер	до 200 строк	не регламентирован

Дополнительно у Claude Code есть `.claude/rules/*.md` с YAML-фронтматтером `paths:` — правила, которые подгружаются, только когда агент трогает подходящие файлы. Это штатный способ разгрузить `CLAUDE.md` в большой репе.

Публичные примеры в open source

Все файлы ниже открыты и проверены; размеры взяты из GitHub API, тип `symlink` — из git-дерева (`mode 120000`).

РЕПОЗИТОРИЙ	ФАЙЛ	ЧЕМ ИНТЕРЕСЕН
<code>apache/airflow</code>	<code>AGENTS.md</code> , 35 КБ	эталон жёстких правил: писать «Dag» в прозе, <code>pytest</code> — только через <code>Breeze</code> , никогда на хосте. <code>CLAUDE.md</code> — симлинк
<code>vercel/next.js</code>	<code>AGENTS.md</code> , 25 КБ	тот же приём симлинка в очень большой монорепе
<code>astral-sh/uv</code>	<code>AGENTS.md</code> , 1.8 КБ	противоположный полюс: два десятка строк <code>ALWAYS/NEVER/PREFER/AVOID</code> . <code>CLAUDE.md</code> — 11 байт, ровно <code>@AGENTS.md</code>
<code>obra/superpowers</code>	<code>CLAUDE.md</code> , 8.7 КБ	связка в обратную сторону: симлинк — <code>AGENTS.md</code>
<code>openai/codex</code>	<code>AGENTS.md</code> , 22 КБ	322 строки про Rust, TUI и code review — выброс, а не образец
<code>microsoft/vscode</code>	<code>AGENTS.md</code> , 271 байт	минимализм: отсылка к <code>.github/copilot-instructions.md</code>
<code>anomalyco/opencode</code>	<code>AGENTS.md</code> , 8.7 КБ	стиль-гайд с примерами «Good / Bad». Бывший <code>sst/opencode</code> , ветка <code>dev</code>
<code>humanlayer/humanlayer</code>	<code>CLAUDE.md</code> , 3.2 КБ	карта монорепы: компоненты, схема потока, make-команды
<code>temporalio/sdk-java</code>	<code>AGENTS.md</code> , 2.3 КБ	как выглядит файл для Java/Gradle
<code>modelcontextprotocol/servers</code>	<code>CLAUDE.md</code> , 4.2 КБ	<code>npm workspaces</code> : <code>build/test</code> -команды на каждый пакет

Отдельно: у `anthropics/claude-code` **нет** ни `CLAUDE.md` , ни `AGENTS.md` в корне, хотя многие подборки рекомендуют его как «канонический пример». Проверено — файлы отдают 404.

Полезные подборки

ПОДБОРКА	ЗВЁЗД	ПРО ЧТО
obra/superpowers	271.1k	фреймворк скиллов и методология; <code>/plugin install superpowers@claude-plugins-official</code>
anthropics/skills	168.4k	официальные скиллы, спека в <code>spec/</code> , шаблон в <code>template/</code> ; <code>/plugin marketplace add anthropics/skills</code>
punkpeye/awesome-mcp-servers	92.1k	самый большой каталог MCP-серверов
modelcontextprotocol/servers	89.5k	референсные MCP-серверы от самого проекта
hesreallyhim/awesome-claude-code	52.2k	ручная подборка команд, хуков, workflow
VoltAgent/awesome-claude-code-subagents	24.2k	100+ готовых субагентов
BehiSecc/awesome-claude-skills	9.9k	каталог скиллов
josix/awesome-claude-md	551	примеры <code>CLAUDE.md</code> плюс сайт-витрина
disler/claude-code-hooks-mastery	3.9k	разбор хуков; без правок с 04.03.2026 — сверяйтесь с документацией

Два популярных списка мертвы и в закладки не годятся: [tairov/awesome-agents.md](#) и [appcypher/awesome-mcp-servers](#) — оба переведены в архив.

Ссылки

URL	ЧТО ЭТО	ПОСЛЕДНЯЯ АКТИВНОСТЬ
https://code.claude.com/docs/en/overview	Документация Claude Code	живая, 200
https://code.claude.com/docs/en/setup	Установка и каналы обновления	живая, 200
https://code.claude.com/docs/en/cli-reference	Справочник флагов CLI	живая, 200
https://code.claude.com/docs/en/memory	CLAUDE.md, <code>.claude/rules/</code> , мост к AGENTS.md	живая, 200
https://code.claude.com/docs/en/changelog	Релиз-ноты; верх — 2.1.228	11.08.2026
https://code.claude.com/docs/llms.txt	Индекс документации для агента	живая, 200
https://github.com/anthropics/claude-code	Issue tracker, CHANGELOG, плагины (141.2k★)	11.08.2026
https://github.com/openai/codex	Codex CLI, Apache-2.0 (105.5k★)	12.08.2026
https://learn.chatgpt.com/docs/codex/cli	Документация Codex CLI, новый адрес	живая, 200
https://learn.chatgpt.com/docs/non-interactive-mode	<code>codex</code> <code>exec</code> : флаги, JSONL, resume	живая, 200
https://agents.md/	Спецификация AGENTS.md	живая, 200
https://github.com/agentsmd/agents.md	Репозиторий спецификации (23.6k★)	12.03.2026
https://aaif.io/projects	AAIF: MCP, goose, AGENTS.md, agentgateway	живая, 200
https://github.com/apache/airflow/blob/main/AGENTS.md	Пример: жёсткие правила окружения	12.08.2026
https://github.com/vercel/next.js/blob/canary/AGENTS.md	Пример: монорепа + симлинк	07.08.2026
https://github.com/astral-sh/uv/blob/main/AGENTS.md	Пример: ALWAYS/NEVER, 1.8 КБ	05.08.2026
https://github.com/astral-sh/uv/blob/main/CLAUDE.md	Мост через импорт <code>@AGENTS.md</code>	05.08.2026
https://github.com/openai/codex/blob/main/AGENTS.md	Пример: 322 строки, Rust	08.07.2026
https://github.com/microsoft/vscode/blob/main/AGENTS.md	Пример: файл-указатель, 271 байт	07.08.2026
https://github.com/anomalyco/opencode/blob/dev/AGENTS.md	Пример: стиль-гайд Good/Bad	25.06.2026
	Пример: карта монорепа	30.09.2025

URL	ЧТО ЭТО	ПОСЛЕДНЯЯ АКТИВНОСТЬ
https://github.com/humanlayer/humanlayer/blob/main/CLAUDE.md		
https://github.com/temporalio/sdk-java/blob/master/AGENTS.md	Пример для Java/Gradle	23.06.2026
https://github.com/modelcontextprotocol/servers/blob/main/CLAUDE.md	Пример: npm workspaces	27.03.2026
https://github.com/obra/superpowers	Фреймворк скиллов (271.1k★)	12.08.2026
https://github.com/anthropics/skills	Официальные скиллы и спека (168.4k★)	07.08.2026
https://github.com/punkpeye/awesome-mcp-servers	Каталог MCP-серверов (92.1k★)	03.08.2026
https://github.com/modelcontextprotocol/servers	Референсные MCP-серверы (89.5k★)	10.08.2026
https://github.com/hesreallyhim/awesome-claude-code	Команды, хуки, workflow (52.2k★)	12.08.2026
https://github.com/VoltAgent/awesome-claude-code-subagents	100+ субагентов (24.2k★)	12.08.2026
https://github.com/BehiSecc/awesome-claude-skills	Каталог скиллов (9.9k★)	02.08.2026
https://github.com/disler/claude-code-hooks-mastery	Хуки (3.9k★), устаревает	04.03.2026
https://github.com/josix/awesome-claude-md	Примеры CLAUDE.md (551★)	05.06.2026

2. Скиллы

компетенция агента: устройство, установка, свои скиллы

Скилл — папка с файлом **SKILL.md**, в котором лежат инструкции для агента. Никакого рантайма и регистрации: положил папку в нужное место — агент её увидел. Формат вынесен в открытый стандарт Agent Skills (agentskills.io) и поддерживается не только Claude Code, но и Codex, Cursor, Gemini CLI, OpenCode, Copilot.

Скилл нужен там, где вы третий раз копируете в чат одну процедуру, или где кусок **CLAUDE.md** разросся из факта в инструкцию. Разница принципиальная: **CLAUDE.md** в контексте всегда, тело скилла — только когда понадобилось.

Структура папки

```
my-skill/
├── SKILL.md           # обязательно: frontmatter + инструкции
├── scripts/          # исполняемый код, который агент запускает
├── references/        # документация, читается по необходимости
└── assets/           # шаблоны, схемы, данные
```

Три подпапки — соглашение спецификации, а не требование. Важнее другое: файл, на который нет ссылки из SKILL.md, агент, скорее всего, не откроет никогда.

Frontmatter

Спецификация определяет шесть полей, обязательны два:

ПОЛЕ	ОБЯЗ.	ОГРАНИЧЕНИЯ
name	да	до 64 символов; a-z , 0-9 , дефис; не начинается/кончается дефисом, без -- ; совпадает с именем папки
description	да	до 1024 символов; что делает и когда применять
license	нет	название лицензии или ссылка на файл
compatibility	нет	до 500 символов, требования к окружению
metadata	нет	произвольная карта «строка → строка»
allowed-tools	нет	предодобренные инструменты (помечено экспериментальным)

Claude Code принимает эти шесть плюс свои расширения: when_to_use, argument-hint, arguments, disable-model-invocation, user-invocable, disallowed-tools, model, effort, context, agent, background, hooks, paths, shell.

Два расхождения, о которых надо знать. Первое: документация Claude Code пишет, что все поля опциональны, рекомендуется только description, а name в личном и проектном скилле задаёт лишь подпись в списке — команда берётся из имени папки. Спецификация требует name обязательно. Практика: локально можно без него, но если скилл поедет на claude.ai, в Skills API или в package_skill.py — оставляйте строго шесть спецификационных полей, иначе паковка падает с Unexpected key(s) in SKILL.md frontmatter. Второе: спецификация ограничивает description 1024 символами, Claude Code режет пару description + when_to_use на 1536. Ориентируйтесь на меньшее.

Где лежат скиллы

уровень	путь	область
Enterprise	через managed settings	вся организация
Личный	~/.claude/skills/<skill-name>/ SKILL.md	все ваши проекты
Проектный	.claude/skills/<skill-name>/ SKILL.md	этот репозиторий
Плагинный	<plugin>/skills/<skill-name>/ SKILL.md	где плагин включён

При совпадении имён enterprise перебивает личный, личный — проектный, любой из них — bundled-скилл Claude Code. Плагинные живут в пространстве имён plugin-name:skill-name и не конфликтуют ни с чем: отсюда /frontend-design:frontend-design в меню.

Проектные подхватываются из .claude/skills/ стартовой директории и всех родительских до корня репозитория. Вложенные (apps/web/.claude/skills/) при старте не грузятся — появляются, когда агент впервые прочитал файл в этой подпапке, и при конфликте имён получают квалифицированное имя /apps/web:deploy (v2.1.203+). Правки подхватываются на лету, перезапуск нужен только если сама директория не существовала на старте. Записи в ~/.claude/skills/ могут быть симлинками — удобно держать скиллы в отдельном git-репозитории.

Progressive disclosure

Загрузка идёт в три стадии:

- 1. **Discovery** — на старте в системный промпт попадают только name и description каждого скилла (~100 токенов на скилл).
- 2. **Activation** — задача совпала с описанием (или вы набрали /skill-name) — читается тело SKILL.md целиком. Потолок: до 5000 токенов, до 500 строк.
- 3. **Execution** — файлы из подпапок читаются по факту нужды. Скрипт, который агент запускает, в контекст вообще не попадает — только его вывод.

Отсюда правило: SKILL.md — оглавление, а не энциклопедия. Ссылки на вспомогательные файлы держите на одном уровне вложенности: при цепочке SKILL.md → a.md → b.md агент читает промежуточные файлы кусками (head -100) и до сути не доходит.

Решение «подгружать или нет» модель принимает по description. Поэтому оно пишется в третьем лице, содержит и «что делает», и «когда применять», и включает слова, которыми пользователь реально формулирует запрос.

Репозиторий anthropics/skills

168 449 звёзд, последний push 2026-08-07 (проверено 2026-08-12). Структура: skills/, spec/, template/ (пустой шаблон на 140 байт), .claude-plugin/.

В skills/ — 17 скиллов: algorithmic-art, brand-guidelines, canvas-design, claude-api, doc-coauthoring, docx, frontend-design, internal-comms, mcp-builder, pdf, pptx, skill-creator, slack-gif-creator, theme-factory, web-artifacts-builder, webapp-testing, xlsx.

```
/plugin marketplace add anthropics/skills
/plugin install document-skills@anthropic-agent-skills
/plugin install example-skills@anthropic-agent-skills
```

Лицензии разные: документные (`docx`, `pdf`, `pptx`, `xlsx`) — source-available, остальные Apache 2.0; на уровне репозитория GitHub API отдаёт `license: null`, так что смотрите `LICENSE` внутри конкретного скилла. README отдельно предупреждает: «for demonstration and educational purposes only».

frontend-design: почему бледная страница стала приложением

Скилл лежит в двух местах с идентичным содержимым: в `anthropics/skills` и как плагин официального маркетплейса. Frontmatter — три строки (`name`, `description`, `license`). Тело — около 1300 слов текста, ни строки кода. Механика в четырёх приёмах, и это рабочий шаблон для ваших скиллов.

Роль вместо задачи. Первая фраза: подходи как арт-директор небольшой студии, известной тем, что каждому клиенту делает идентичность, которую нельзя спутать ни с чьей другой; клиент уже отверг шаблонные предложения. Модель получает не ТЗ, а планку и запрет на средний ответ.

Явная калибровка дефолтов. Скилл прямым текстом перечисляет три вида, вокруг которых кучкуется нынешний AI-дизайн: кремовый фон около `#F4F1EA` с контрастным серифом и терракотовым акцентом; почти чёрный фон с одним кислотно-зелёным или вермильоновым акцентом; broadsheet-вёрстка с волосяными линейками, нулевым скруглением и газетными колонками. Это самая ценная часть файла: без неё модель не знает, что её «красиво по умолчанию» и есть шаблон. Оговорка прописана: если бриф явно просит такой вид — бриф выигрывает.

Два прохода вместо одного. Сначала система токенов: палитра из 4–6 именованных hex, минимум две гарнитуры по ролям (характерный `display` + нейтральный `body`, при нужде `utility` для подписей и цифр), концепт лейаута прозой и ASCII-вайрфреймами, и один `signature`-элемент, по которому страницу запомнят. Затем — критика плана против брифа: если часть плана выглядит так, будто вы выдали бы её на любой похожий запрос, её переписывают и объясняют, что изменилось. Только после этого пишется код, и каждый цвет и кегль выводится из плана.

Дисциплина и потолок качества. Смелость тратится в одном месте, остальное остаётся тихим. Планка без обсуждения: адаптив до мобильного, видимый фокус с клавиатуры, уважение `prefers-reduced-motion`. Отдельно — предупреждение про специфичность CSS-селекторов (классы `.section` и `.cta` взаимно гасят отступы) и цитата Шанель про «сними одно украшение перед выходом». Плюс раздел про копирайтинг: активный залог, «Save changes» вместо «Submit», кнопка `Publish` даёт тост `Published`.

Результат меняется не потому, что скилл добавил библиотеку, а потому, что он задал роль, назвал запрещённые дефолты, вставил обязательную критику между планом и кодом и зафиксировал нижнюю границу качества.

Установка

Маркетплейс. `claude-plugins-official` добавляется автоматически при первом интерактивном запуске; на 2026-08-12 в каталоге 287 плагинов. Если нет:

```
/plugin marketplace add anthropics/claude-plugins-official
/plugin install frontend-design@claude-plugins-official
/reload-plugins
```

Сводка установки пишет `Plugin is now active.` — перезагрузка не нужна; `Run /reload-plugins to activate.` — выполните команду.

Вручную:

```
mkdir -p ~/.claude/skills/my-skill  
$EDITOR ~/.claude/skills/my-skill/SKILL.md
```

Из чужого репозитория. Есть `.claude-plugin/marketplace.json` — `/plugin marketplace add owner/repo`, дальше `/plugin install <name>@<marketplace>`. Поддерживаются полные git-URL (GitLab, Bitbucket, self-hosted), локальные пути, `#ref` для ветки или тега. Если это просто набор папок — клонируйте и симлинкуйте в `~/.claude/skills/`. Плагины исполняют произвольный код с вашими правами: `claude plugin details <name>` показывает состав и стоимость в токенах до установки.

Проверка. Меню `/skills`, вопрос «What skills are available?», строка Skills в `/context`, оценка листинга в `/doctor`. При невалидном YAML тело загрузится с пустыми метаданными: `/skill-name` сработает, автоподхват — нет; ошибку парсинга видно с `claude --debug`.

Свой скилл под свой стек

Порядок: пройдите задачу обычным промптингом, заметьте, какой контекст даёте руками повторно, попросите Claude собрать из этого скилл (формат модели знают нативно), затем вычистите объяснения того, что модель и так знает.

```

----
# имя папки; в личном/проектном скилле — только подпись в списке,
# команда всё равно берётся из директории
name: reviewing-swift-concurrency

# третье лицо, "что делает" + "когда применять" + слова пользователя.
# ключевой кейс первым: при переполнении бюджета описание режут с конца
description: Reviews Swift code for actor isolation, Sendable conformance and
  structured concurrency mistakes. Use when the compiler warns about data races,
  when adding async/await to existing code, or when the user mentions actors,
  @MainActor or Sendable.

paths: "**/*.swift"                                # ограничить автоподхват
allowed-tools: Bash(swift build *) Read Grep      # действует на текущий ход
----

# Swift concurrency review

## Когда применять
Короткий триаж: что сюда попадает, а что нет.

## Процедура
1. Первый шаг с точной командой.
2. Второй шаг.
3. Валидация: `swift build -Xswiftc -strict-concurrency=complete`.
   Дальше не идти, пока не чисто.

## Правила проекта
Только то, чего модель знать не может: ваши конвенции и запреты.
Не пересказывать документацию Swift.

## Дополнительно
- Таблица правил изоляции: [references/isolation.md](references/isolation.md)
- Примеры до/после: [references/examples.md](references/examples.md)

```

В теле: соразмеряйте свободу задаче (где вариантов много — общее направление, где операция хрупкая — «выполни ровно эту команду, не меняй флаги»); давайте петлю обратной связи «валидатор → починить → повторить» с явным «дальше не идти»; приводите примеры вход/выход, если качество зависит от стиля; держите один термин на одно понятие.

Типичные ошибки. Расплывчатое описание («helps with documents») — скилл не подхватится никогда. Описание от первого лица — портит выбор, оно идёт прямо в системный промпт. Пересказ известного модели. Многоуровневые ссылки. Даты вида «до августа делай так» — вместо этого раздел «Old patterns» под `<details>`. Windows-пути с обратными слэшами. Меню из пяти библиотек вместо одного дефолта с оговоркой. `context: fork` на скилле-справочнике: субагент получит правила без задачи и вернётся ни с чем. Магические константы в скриптах (`TIMEOUT = 47`) без объяснения.

Тестирование. Факт срабатывания ничего не доказывает. Проверяются две вещи порознь: подхватывается ли на нужных запросах и лучше ли результат. Метод — сравнение с базой: несколько реалистичных промптов, каждый в чистой сессии со скиллом и без (отключается через `skill0verrides: {"my-skill": "off"}`). Свежая сессия обязательна: остатки контекста маскируют дыры в инструкциях. Цикл автоматизирует `/plugin install skill-creator@claude-plugins-official:` кейсы в `evals/evals.json`, каждый прогон в отдельном субагенте, `grading.json` с PASS/FAIL и доказательством, `benchmark.json` с дельтой — насколько выросла доля прохождений и сколько это стоило в токенах и секундах.

Стоимость в контексте

Всегда в контексте — только листинг: имена плюс описания. Бюджет листинга — **1% контекстного окна модели**; регулируется настройкой `skillListingBudgetFraction` (`0.02 = 2%`) или переменной `SLASH_COMMAND_TOOL_CHAR_BUDGET`. Каждая запись отдельно ограничена 1536 символами (`skillListingMaxDescChars`). При переполнении описания режутся, начиная с самых редко используемых; имена остаются всегда. Размер после применения бюджета — строка Skills в `/context`, разбор по вкладчикам — `/doctor`.

Тело грузится при вызове и **остаётся в контексте до конца сессии**: Claude Code не перечитывает файл на следующих ходах. Пишите как постоянные указания, а не разовые шаги. Повторный вызов с тем же содержимым не дублирует текст (с v2.1.202). При автокомпрессии инвокации переносятся за суммаризацию с бюджетом: по 5000 первых токенов на скилл, суммарно 25 000, заполняется от самого свежего вызова — старые выпадают полностью, их надо вызвать заново.

Не платить за неиспользуемое: `disable-model-invocation: true` убирает описание из контекста целиком (скилл остаётся по `/name`); `skill0verrides: {"x": "name-only"}` оставляет имя без описания; `"off"` прячет совсем. Файлы в `references/`, `assets/`, `scripts/` не стоят ничего, пока их не прочитали — большие справочники смело бандлите.

Ссылки

URL	ЧТО ЭТО	ПОСЛЕДНЯЯ АКТИВНОСТЬ
https://code.claude.com/docs/en/skills	Документация Claude Code: frontmatter, пути, подстановки, lifecycle	Проверено 2026-08-12, описывает поведение до v2.1.227
https://agentskills.io	Открытый стандарт: progressive disclosure, список клиентов	Проверено 2026-08-12
https://agentskills.io/specification	Спецификация формата: поля, ограничения, подпапки	Проверено 2026-08-12
https://agentskills.io/skill-creation/evaluating-skills	Формат <code>evals.json</code> , прогон с базой, ассерты, бенчмарк	Проверено 2026-08-12
https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices	Практики написания, антипаттерны, чеклист	Проверено 2026-08-12
https://github.com/anthropics/skills	Репозиторий Anthropic: 17 скиллов, спецификация, шаблон	168 449 звёзд, push 2026-08-07
https://github.com/anthropics/skills/blob/main/skills/frontend-design/SKILL.md	Исходник разобранного frontend-design	В составе репозитория выше
https://github.com/anthropics/claude-plugins-official	Официальный маркетплейс, 287 записей в каталоге	33 448 звёзд, push 2026-08-12
https://github.com/anthropics/claude-plugins-official/tree/main/plugins/skill-creator	Плагин skill-creator: создание, оценка, бенчмарк скиллов	В составе маркетплейса выше
https://github.com/agentskills/agentskills	Репозиторий стандарта: документация, skills-ref	24 178 звёзд, push 2026-08-09
https://code.claude.com/docs/en/discover-plugins	Установка и управление плагинами и маркетплейсами	Проверено 2026-08-12

3. superpowers и Spec-Driven Development

флоу «спека → план → код → проверка»

Что это такое

Superpowers — плагин для кодинг-агентов, который навязывает агенту методологию разработки: не бросаться писать код, а сначала выяснить задачу, записать спеку, разложить её в план и только потом реализовывать — с ревью на каждом шаге.

Автор — **Jesse Vincent**, GitHub-ник **obra**, вместе с командой Prime Radiant. Это подтверждается и манифестом плагина (author.name: "Jesse Vincent", jesse@fsck.com), и README, и анонсом релиза от 9 октября 2025.

ПАРАМЕТР	ЗНАЧЕНИЕ (ПРОВЕРЕНО 12.08.2026)
Репозиторий	<code>github.com/obra/superpowers</code>
Лицензия	MIT
Создан	2025-10-09
Последняя активность	2026-08-12 (push в тот же день)
Последний релиз	v6.2.0 , опубликован 2026-07-24
Звёзды / форки	~271 000 / ~24 200 (GitHub API)

Цифра в 271k звёзд выглядит аномально большой для репозитория возрастом меньше года, но её одинаково отдают и REST API GitHub, и страница репозитория; витрина плагина на claude.com показывает более миллиона установок — порядок сходится.

Установка

Для Claude Code есть два маркетплейса, оба рабочие:

```
# Вариант 1 – официальный маркетплейс Anthropic (зарегистрирован по умолчанию)
/plugin install superpowers@claude-plugins-official

# Вариант 2 – маркетплейс автора
/plugin marketplace add obra/superpowers-marketplace
/plugin install superpowers@superpowers-marketplace
```

Superpowers существует не только для Claude Code: README документирует установку для Antigravity, Codex App, Codex CLI, Cursor, Factory Droid, Gemini CLI, GitHub Copilot CLI, Kimi Code, OpenCode и Pi.

Важное уточнение про команды. В версии 6.2.0 в плагине **нет каталога `commands/`** — только `skills/`, `hooks/`, `scripts/`, `docs/`, `tests/`. Слэш-команды вида `/brainstorm`, `/write-plan`, `/execute-plan` из старых статей и с витрины плагина на claude.com — устаревшая информация. В Claude Code скиллы плагина автоматически получают неймспейс, поэтому вызывать их вручную нужно так:

```
/superpowers:brainstorming
/superpowers:writing-plans
/superpowers:systematic-debugging
```

Состав: 14 скиллов версии 6.2.0

скилл	что делает
<code>using-superpowers</code>	Бутстрап: требует проверить наличие подходящего скилла до любого ответа, включая уточняющие вопросы
<code>brainstorming</code>	Сократический диалог по одному вопросу за раз; на выходе — согласованная спека
<code>writing-plans</code>	Превращает спеку в план из шагов по 2–5 минут с точными путями и кодом
<code>executing-plans</code>	Последовательное выполнение плана в текущей сессии с чекпоинтами для человека
<code>subagent-driven-development</code>	Свежий сабагент на каждую задачу + двухстадийное ревью; рекомендуемый режим
<code>test-driven-development</code>	Жёсткий RED-GREEN-REFACTOR; код, написанный до теста, удаляется
<code>systematic-debugging</code>	Четырёхфазный поиск корневой причины до предложения фикса
<code>verification-before-completion</code>	Запрещает говорить «готово» без вывода реально запущенной проверки
<code>requesting-code-review</code>	Диспатчит ревьюера-сабагента с узко собранным контекстом
<code>receiving-code-review</code>	Как отвечать на замечания: проверять, а не соглашаться из вежливости
<code>using-git-worktrees</code>	Изолированное рабочее пространство на отдельной ветке
<code>finishing-a-development-branch</code>	Финал: merge / PR / оставить / выбросить
<code>dispatching-parallel-agents</code>	Параллельные сабагенты для 2+ независимых задач
<code>writing-skills</code>	Мета-скилл: как писать и тестировать собственные скиллы

Внутри скиллов лежат вспомогательные файлы: у `systematic-debugging` — `root-cause-tracing.md`, `defense-in-depth.md`, `condition-based-waiting.md`, скрипт `find-polluter.sh`; у `test-driven-development` — `writing-good-tests.md`. Маркетплейс автора содержит ещё три плагина: `elements-of-style`, `superpowers-developing-for-claude-code` и `private-journal-mcp`.

Механика автозапуска

Ключ ко всему — **один SessionStart-хук**. Файл `hooks/hooks.json` в плагине:

```
{
  "hooks": {
    "SessionStart": [{
      "matcher": "startup|clear|compact",
      "hooks": [{
        "type": "command",
        "command": "\"\\${CLAUDE_PLUGIN_ROOT}/hooks/run-hook.cmd\" session-start",
        "shell": "bash", "async": false
      }]
    }]
  }
}
```

Скрипт `hooks/session-start` читает **полный текст** `skills/using-superpowers/SKILL.md`, экранирует его в JSON и отдаёт через `hookSpecificOutput.additionalContext` — штатный механизм Claude Code для инъекции контекста. Текст оборачивается в тег `<EXTREMELY_IMPORTANT>` со словами «You have superpowers».

Матчер `startup|clear|compact` означает: инструкция подмешивается при старте, после `/clear` и **после компакции контекста** — то есть переживает сжатие истории. Скрипт определяет платформу по переменным окружения (`CURSOR_PLUGIN_ROOT`, `CLAUDE_PLUGIN_ROOT`, `COPILLOT_CLI`) и выбирает нужный формат поля.

Сам `using-superpowers` — это давление промптом, а не код: «если есть хотя бы 1% шанс, что скилл применим, ты ОБЯЗАН его вызвать», плюс таблица «красных флагов» с типовыми отговорками («это просто вопрос», «сначала осматриваю репозиторий») и их опровержениями. Приоритет прописан явно: процессные скиллы идут первыми, реализационные — после. И отдельной строкой: инструкции пользователя (`CLAUDE.md`, `AGENTS.md`, прямые просьбы) **выше** скиллов.

Флоу SDD по шагам

1. Brainstorming → спецификация

Активируется на любую формулировку «давай сделаем X». Жёсткий гейт: `Do NOT invoke any implementation skill, write any code... until you have presented a design and the user has approved it` — и отдельная секция против отговорки «это слишком просто для дизайна».

Чек-лист из 9 пунктов: изучить контекст проекта → задавать уточняющие вопросы **по одному за сообщение** → предложить 2–3 подхода с трейд-оффами и рекомендацией → показать дизайн секциями, спрашивая одобрение после каждой → записать документ → само-ревью → ревью человеком → перейти к плану.

Артефакт: `docs/superpowers/specs/YYYY-MM-DD-<topic>-design.md`, коммитится в git. Само-ревью спеки проверяет четыре вещи: заглушки TBD/TODO, внутренние противоречия, размер скоупа, двусмысленные требования.

Где вмешивается человек: одобряет каждую секцию дизайна, затем читает записанный файл и даёт добро. Формулировка гейта прямо в скилле: «*Spec written and committed to <path>. Please review it...*»

У brainstorming есть визуальный компаньон — браузерная вкладка с макетами и диаграммами. Скилл требует предлагать его **не заранее**, а в момент, когда вопрос действительно проще показать, чем описать.

2. Writing-plans → план

Правило написания плана: считать исполнителя грамотным разработчиком, который ничего не знает о вашем домене и плохо владеет дизайном тестов. Шаг = одно действие на 2–5 минут: написать падающий тест → убедиться, что падает → минимальная реализация → убедиться, что проходит → КОММИТ.

Артефакт: docs/superpowers/plans/YYYY-MM-DD-<feature-name>.md .

В шапке плана обязательны Goal, Architecture, Tech Stack и блок **Global Constraints** — версии, ограничения зависимостей, правила именования, скопированные из спеки дословно. Каждая задача несёт блок **Interfaces** (Consumes / Produces с точными сигнатурами), потому что исполнитель задачи видит только свою задачу.

Секция **No Placeholders** перечисляет формулировки, считающиеся дефектом плана: «TBD», «add appropriate error handling», «write tests for the above» без кода тестов, «similar to Task N».

3. Реализация

После сохранения плана скилл предлагает выбор из двух режимов:

РЕЖИМ	КАК РАБОТАЕТ	КОГДА
subagent-driven-development (рекомендуемый)	Свежий сабагент на задачу, ревью после каждой (соответствие спеке + качество кода), широкое ревью всей ветки в конце	Есть сабагенты, задачи независимы
executing-plans	Выполнение в текущей сессии батчами с чекпоинтами для человека	Нет сабагентов или задачи сильно связаны

Перед стартом обоих режимов вызывается **using-git-worktrees** — изолированная ветка и чистый базовый прогон тестов. **subagent-driven-development** намеренно **не спрашивает человека между задачами**: останов только на блокере, реальной неоднозначности или завершении плана.

4. Независимая проверка

requesting-code-review диспатчит **general-purpose** сабагента по шаблону **code-reviewer.md**, передавая ему не историю сессии, а собранный вручную контекст:

```
BASE_SHA=$(git rev-parse HEAD~1) # или origin/main
HEAD_SHA=$(git rev-parse HEAD)
```

Ревьюер возвращает находки по severity: Critical чинится немедленно, Important — до следующей задачи, Minor фиксируется на потом. **verification-before-completion** запрещает объявлять работу готовой без вывода реально выполненной команды. Финал — **finishing-a-development-branch**: прогон тестов и выбор merge / PR / оставить ветку / выбросить.

Альтернативы

ПРОЕКТ	МОДЕЛЬ РАБОТЫ	ЗВЁЗДЫ / СТАТУС (12.08.2026)
GitHub Spec Kit	CLI <code>specify</code> + слэш-команды <code>/speckit.constitution</code> → <code>/speckit.specify</code> → <code>/speckit.plan</code> → <code>/speckit.tasks</code> → <code>/speckit.implement</code> , плюс <code>/speckit.clarify</code> и <code>/speckit.analyze</code> для сверки артефактов. 30+ агентов, MIT	~126 400, активен
Kiro (AWS)	Отдельная IDE на Code OSS. Генерирует <code>requirements.md</code> (нотация EARS), <code>design.md</code> , <code>tasks.md</code> в <code>.kiro/specs/</code> с гейтами между фазами; режим Quick Spec гейты пропускает	Проприетарный: free 50 кредитов, Pro \$20, Pro+ \$40, Pro Max \$100, Power \$200 в месяц
OpenSpec	<code>npm i -g @fission-ai/openspec</code> + <code>openspec init</code> ; цикл <code>/opsx:explore</code> → <code>/opsx:propose</code> → <code>/opsx:apply</code> → <code>/opsx:archive</code> . Лёгкий слой поверх существующей кодовой базы, MIT	~64 600, активен
BMAD-METHOD	<code>npx bmad-method install</code> ; симуляция команды из ролей (product, architecture, UX, dev, QA), цикл Clarify → Plan → Build & verify → Learn, MIT	~51 800, активен
Agent OS	Инъекция стандартов кодовой базы, а не долгоживущие спеки	~5 300, последняя активность 05.05.2026

Разница в философии: Spec Kit и Kiro дают **явные команды** и жёсткую последовательность документов; Superpowers прячет всё за автосрабатыванием скиллов и делает ставку на диалог и TDD. Spec Kit ближе, если нужен формальный артефакт-контракт для команды; Superpowers — если нужно, чтобы агент сам не срывался в код.

Практика: воспроизвести флоу у себя

```
# 1. Установка (в сессии Claude Code)
/plugin install superpowers@claude-plugins-official
/reload-plugins          # если установщик попросил

# 2. Проверка, что хук отработал: в начале новой сессии
#   в контекст подмешан текст using-superpowers.
#   Убедиться, что скиллы видны:
/context

# 3. Новая фишка — просто описать задачу словами.
#   brainstorming включится сам; если нет — позвать явно:
/superpowers:brainstorming добавить экспорт отчётов в CSV

# 4. Отвечать на вопросы по одному. Одобрять секции дизайна.
#   Прочитать записанный файл:
#   docs/superpowers/specs/2026-08-12-csv-export-design.md

# 5. После одобрения спеки агент вызовет writing-plans.
#   Проверить план глазами:
#   docs/superpowers/plans/2026-08-12-csv-export.md

# 6. Выбрать режим исполнения — 1 (subagent-driven) или 2 (inline).

# 7. Изолированная ветка создаётся скиллом using-git-worktrees:
git worktree list

# 8. Финал — finishing-a-development-branch предложит merge/PR.
```

Что проверять человеку: в спеке — нет ли «TBD» и двусмысленных требований; в плане — есть ли у каждого требования спеки своя задача и совпадают ли имена функций между задачами; после каждой задачи — реальный вывод тестов, а не утверждение «всё работает».

Телеметрия: логотип Prime Radiant в визуальном компаньоне brainstorming грузится с сайта авторов и передаёт версию Superpowers (не передаёт содержимое проекта и промпты). Отключается переменной `SUPERPOWERS_DISABLE_TELEMETRY`, уважает также `DISABLE_TELEMETRY` и `CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC`.

Ссылки

URL	ЧТО ЭТО	ПОСЛЕДНЯЯ АКТИВНОСТЬ
https://github.com/obra/superpowers	Основной репозиторий, MIT, ~271k звёзд, релиз v6.2.0	push 12.08.2026, релиз v6.2.0 от 24.07.2026
https://blog.fsck.com/2025/10/09/superpowers/	Анонс релиза от Jesse Vincent, описание идеи скиллов	09.10.2025
https://claude.com/plugins/superpowers	Витрина плагина в официальном каталоге Claude (упоминание слэш-команд там устарело)	актуальна на 12.08.2026
https://github.com/obra/superpowers-marketplace	Маркетплейс автора: superpowers, elements-of-style, developing-for-claude-code, private-journal-mcp	push 07.08.2026
https://code.claude.com/docs/en/plugins	Документация Claude Code: состав плагина, неймспейс <code>/plugin-name:skill-name</code>	актуальна на 12.08.2026
https://code.claude.com/docs/en/hooks	Документация: событие SessionStart, матчеры, <code>hookSpecificOutput.additionalContext</code>	актуальна на 12.08.2026
https://github.com/github/spec-kit	GitHub Spec Kit, MIT, ~126k звёзд, команды <code>/speckit.*</code>	push 12.08.2026
https://kiro.dev/docs/specs/	Документация Kiro: requirements.md / design.md / tasks.md, гейты, Quick Spec	актуальна на 12.08.2026
https://kiro.dev/pricing/	Тарифы Kiro: free 50 кредитов, \$20 / \$40 / \$100 / \$200	актуальна на 12.08.2026
https://github.com/Fission-AI/OpenSpec	OpenSpec, MIT, ~64,6k звёзд, цикл <code>/opsx:*</code>	push 12.08.2026
https://github.com/bmad-code-org/BMAD-METHOD	BMAD-METHOD, MIT, ~51,8k звёзд, <code>npx bmad-method install</code>	push 11.08.2026

4. MCP

подключение внешних инструментов и его цена

Что это и кто за этим стоит

MCP — открытый стандарт подключения AI-приложений к внешним системам. Официальный сайт описывает его метафорой: «MCP — это как порт USB-C для AI-приложений».

Протокол создала Anthropic. 9 декабря 2025 года компания передала MCP в Linux Foundation — в новый Agentic AI Foundation (AAIF), где MCP стал проектом-основателем наряду с goose от Block и AGENTS.md от OpenAI. Модель управления не изменилась: «the project's maintainers will continue to prioritize community input and transparent decision-making».

Актуальная версия спецификации — 2026-07-28, опубликована 28 июля 2026; версии здесь датированные, а не семантические. Главное из релиза: stateless-ядро (сессий на уровне протокола

больше нет, каждый запрос самодостаточен), Multi Round-Trip Requests (инструмент может запросить ввод у человека посреди выполнения), маршрутизация по HTTP-заголовкам `Mcp-Method` / `Mcp-Name` , формальный механизм extensions (Tasks, MCP Apps, Skills over MCP) и объявление старого транспорта HTTP+SSE устаревшим с 12-месячным окном миграции.

Формат сообщений — JSON-RPC 2.0. Роли: **Host** (LLM-приложение, например Claude Code), **Client** (коннектор внутри хоста), **Server** (сервис с данными и инструментами).

Транспорты

ТРАНСПОРТ	КАК РАБОТАЕТ	КОГДА ПРИМЕНЯЮТ
stdio	Клиент запускает сервер дочерним процессом, строчный JSON-RPC через stdin/stdout	Локальные инструменты: файлы, БД, браузер
Streamable HTTP	Каждое сообщение — HTTP POST на единый endpoint; ответ либо JSON, либо SSE-поток, привязанный к запросу	Хостируемые сервисы, OAuth

Кастомные транспорты допускаются, но обязаны сохранять JSON-RPC и per-request-метаданные. Claude Code дополнительно поддерживает устаревший `sse` и `ws` (WebSocket добавляется только через `claude mcp add-json` , флаг `--transport` его не принимает).

Tools / Resources / Prompts

ПРИМИТИВ	ЧТО ЭТО	КТО УПРАВЛЯЕТ	МЕТОДЫ
Tools	Функции, которые модель вызывает сама; вход описан JSON Schema	Модель	<code>tools/list</code> , <code>tools/call</code>
Resources	Пассивные данные на чтение, у каждого свой URI и MIME-тип	Приложение	<code>resources/list</code> , <code>resources/read</code>
Prompts	Шаблоны-заготовки, вызываются пользователем явно	Пользователь	<code>prompts/list</code> , <code>prompts/get</code>

Со стороны клиента сервер может использовать **elicitation** — запрос данных у человека посреди задачи. В Claude Code ресурсы подтягиваются через `@server:protocol://path` (`@github:issue://123`), а промпты становятся командами `/mcp__servername__promptname` .

Подключение к Claude Code

```
# HTTP-сервер
claude mcp add --transport http claude-code-docs https://code.claude.com/docs/mcp

# HTTP со статичным токеном
claude mcp add --transport http github https://api.githubcopilot.com/mcp/ \
  --header "Authorization: Bearer YOUR_GITHUB_PAT"

# Локальный stdio-сервер: всё после `--` уходит серверу как есть
claude mcp add playwright -- npx -y @playwright/mcp@latest
```

SCOPE	ФАЙЛ	КОМУ ДОСТУПЕН
local (по умолчанию)	~/.claude.json, внутри записи текущего проекта	только вам, только в этом проекте
project	.mcp.json в корне проекта	всей команде через git
user	~/.claude.json, ключ mcpServers верхнего уровня	только вам, во всех проектах

Scope фиксируется при добавлении: чтобы переехать, нужно `claude mcp remove` и добавить заново с `--scope`. Если сервер описан в нескольких местах, Claude Code берёт запись целиком из источника с высшим приоритетом (поля между scope не смешиваются): local → project → user → серверы плагинов → коннекторы claude.ai. В `.mcp.json` работает подстановка `${VAR}` и `${VAR:-default}` в полях `command`, `args`, `env`, `url`, `headers`.

```
{
  "mcpServers": {
    "claude-code-docs": { "type": "http", "url": "https://code.claude.com/docs/mcp" },
    "playwright": { "type": "stdio", "command": "npx", "args": ["-y", "@playwright/
      mcp@latest"] }
  }
}
```

Просмотр и отладка: `claude mcp list` (статусы `✓ Connected`, `! Needs authentication`, `✗ Failed to connect`, `⏸ Pending approval`), `claude mcp get <name>` (с v2.1.219 показывает HTTP-статус и текст ошибки сервера в строке `Issue:`), `claude mcp remove <name>`. Внутри сессии — `/mcp`: статусы, число инструментов, `Authenticate / Reconnect`, отключение сервера без удаления.

Разбор поломки: для HTTP — `curl -I <url>` (404/405 значит, что сервер жив, многие endpoint'ы принимают только POST; 401/403 — нужна авторизация); для stdio — запустить команду руками, и если процесс ждёт ввода, проблема в конфиге (частая причина — забытый разделитель `--`). Таймаут старта в 30 секунд поднимается через `MCP_TIMEOUT=60000 claude`. Правки `.mcp.json` подхватываются только при старте сессии; однажды отклонённый сервер возвращается командой `claude mcp reset-project-choices`.

Серверы с занятия

Playwright MCP (Microsoft)

36 042 звезды, Apache-2.0, последний push 9 августа 2026; npm `@playwright/mcp` 0.0.79 от 6 августа 2026. Работает не по скриншотам, а по accessibility-дереву страницы — быстрее и дешевле по токенам.

```
claude mcp add playwright -- npx -y @playwright/mcp@latest
```

Базовый набор: `browser_navigate`, `browser_click`, `browser_type`, `browser_fill_form`, `browser_snapshot`, `browser_find`, `browser_console_messages`, `browser_network_requests`.

Тестировщику важнее опциональные наборы через `--caps: testing` даёт `browser_generate_locator` (готовый локатор для теста) и семейство `browser_verify_*`; `network` — мокирование запросов; `storage` — cookies и localStorage; `devtools` — трассировки и видеозапись прогона. Полезные флаги: `--isolated` вместе с `--storage-state=path.json` для авторизованных сессий, `--headless`, `--mobile` (мобильные страницы легче — экономия токенов). Есть образ `mcr.microsoft.com/playwright/mcp`. Два честных предупреждения из README: «Playwright MCP is **not** a security boundary», и что для агентов с большим кодовым контекстом связка Playwright CLI + SKILLS экономнее по токенам, чем MCP.

Context7 (Upstash)

60 643 звезды, MIT, последний push 12 августа 2026; npm `@upstash/context7-mcp` 4.0.2 от 11 августа 2026. Закрывает ровно ту проблему, из-за которой модель уверенно пишет код по API двухлетней давности: подтягивает актуальную документацию конкретной версии библиотеки прямо в промпт. Два инструмента: `resolve-library-id` (имя → идентификатор вида `/vercel/next.js`) и `query-docs` (идентификатор + вопрос → релевантные фрагменты).

```
npx ctx7 setup # OAuth, генерация ключа, установка; флаг --claude для Claude Code
```

Вручную: URL `https://mcp.context7.com/mcp`, ключ в заголовке `Authorization: Bearer YOUR_API_KEY`. Без ключа работает, но с меньшими лимитами. Оговорка из README проекта: документация наполняется сообществом, её точность и безопасность не гарантируются.

Stitch (Google Labs)

AI-канвас для проектирования интерфейсов. Пост Google от 18 марта 2026 подтверждает: «Using the recently released Stitch MCP server and SDK, you can leverage Stitch’s capabilities via skills and tools» — официальный MCP-сервер существует, а не только сторонние обёртки, которых в каталогах несколько. Документация на `stitch.withgoogle.com/docs/mcp/setup/` отвечает 200, но рендерится JavaScript, поэтому **точную команду подключения и список инструментов по официальному источнику подтвердить не удалось** — сверяйтесь с этой страницей вживую, а не с блогами-пересказами.

Pencil

Дизайн-инструмент поверх файлов `.pen`, которые лежат в репозитории рядом с кодом. Ставится расширением VS Code/Cursor или десктоп-приложением; документация требует установленного и авторизованного Claude Code. MCP-сервер отдельно подключать не нужно: «The pen.dev MCP (Model Context Protocol) server runs automatically when you use pen.dev». Домен `pencil.dev` редиректит на `pen.dev`, документация осталась на `docs.pencil.dev` (страница установки обновлена 28 июля 2026). Коммерческий продукт третьей стороны, внешнего аудита не проходил.

Реестр и каталоги

Официальный реестр — `registry.modelcontextprotocol.io`. Это каталог метаданных, а не хостинг: записи указывают на npm, PyPI, Docker-образы и удалённые endpoint’ы. Имена в формате reverse-DNS (`io.github.user/server`) привязаны к подтверждённому GitHub-аккаунту или домену. Публичный API без авторизации: GET `https://registry.modelcontextprotocol.io/v0/servers?limit=1`.

Крупные сторонние каталоги на 12 августа 2026: **PulseMCP** — 22 074 сервера, **Smithery** — «14,281+ MCPs», а также Glama и mcp.so. Цифры между каталогами не сходятся, дедупликация не гарантирована — это порядок величины, не перепись. Anthropic ведёт свой список проверенных коннекторов — Anthropic Directory на claude.ai/directory.

Безопасность

Главный факт: локальный MCP-сервер — это программа, которую клиент запускает подпроцессом с вашими правами и без песочницы. Спецификация формулирует риски прямо: «Arbitrary code execution... Attackers can execute any command with MCP client privileges», «No visibility. Users have no insight into what commands are being executed». Песочница — рекомендация (SHOULD) хостам, а не гарантия протокола.

Prompt injection через данные инструмента. Инструмент возвращает текст, который модель читает как инструкцию. 26 мая 2025 Invariant Labs показали атаку на GitHub MCP: злоумышленник заводит issue в публичном репозитории, пользователь просит агента разобрать issues — агент читает вредоносный текст и утекает содержимое частных репозиториях через PR в публичный. Дыра не в коде сервера, а в архитектуре агента, и патчем на стороне сервера не закрывается.

Вредоносный сервер / rug-pull. Сентябрь 2025, находка Koi Security: npm-пакет `postmark-mcp` копировал официальный сервер Postmark. Первые 15 версий были чистыми, а с 1.0.16 пакет добавлял скрытый BCC на адрес злоумышленника в каждое письмо, отправленное агентом. 1 643 загрузки. Первый публично задокументированный вредоносный MCP-сервер «в дикой природе».

Уязвимости в добросовестных серверах. Yarden Porat (Cyata) нашёл три бага в `mcp-server-git`, эталонной реализации Anthropic: path traversal в `git_init` (CVE-2025-68143), argument injection в `git_diff / git_checkout` (CVE-2025-68144) и обход ограничения `--repository` (CVE-2025-68145). Цепочка давала RCE исключительно через prompt injection — например, из README или описания issue. Исправлено в 2025.9.25 и 2025.12.18, `git_init` удалён совсем.

Уязвимости в самой оснастке. CVE-2025-49596 в MCP Inspector: CVSS 9.4, отсутствие аутентификации между клиентом и прокси. В связке с браузерным «0.0.0.0-day» хватало открыть вредоносную страницу, чтобы получить исполнение кода на машине разработчика. Затронуты версии ниже 0.14.1.

Утечка токенов и OAuth-атаки. Security Best Practices спецификации разбирает confused deputy, token passthrough («MCP servers MUST NOT accept any tokens that were not explicitly issued for the MCP server»), SSRF при discovery метаданных (включая `http://169.254.169.254/`) и `javascript:`-схемы в authorization URL.

Что проверять перед установкой стороннего сервера: кто издатель и совпадает ли он с владельцем сервиса (похожее имя пакета ничего не доказывает); активность и лицензия репозитория, а главное — соответствие исходников на GitHub тому, что лежит в npm/PyPI; какая команда прописывается в конфиг (`claude mcp add ... -- <команда>` пишет её дословно и исполняет при каждом старте сессии); какие токены сервер просит и какого объёма у них права; читает ли сервер неподконтрольный внешний контент — issues, письма, веб-страницы (если да, весь он для модели недоверенный ввод); есть ли инструменты, пишущие наружу, — им нужны явные правила разрешений.

Позиция Anthropic сформулирована без обиняков: «Anthropic reviews connectors against its listing criteria before adding them to the Anthropic Directory, but does not security-audit or manage any MCP server». Наличие сервера в каталоге не означает аудита.

Со стороны Claude Code: серверы из `.mcp.json` требуют явного подтверждения в интерактивной сессии, а с v2.1.196 клонированный репозиторий не может одобрить сам себя, пока вы не приняли диалог

доверия рабочей области. В неинтерактивных режимах (`claude -p` , Agent SDK, облачные сессии) диалога нет и project-scoped серверы грузятся молча — блокировать их надо через `disabledMcpjsonServers` . В правилах разрешений инструменты адресуются как `mcp__<server>__<tool>` , а для серверов из плагинов — `mcp__plugin_<plugin>_<server>__<tool>` . Организациям доступен `managed-mcp.json` со списками `allowedMcpServers` / `deniedMcpServers` .

Стоимость MCP в контексте

Исторически описания всех инструментов подключённых серверов загружались в контекст до первого сообщения пользователя: десяток серверов съедал десятки тысяч токенов ещё до того, как вы что-то спросили.

В Claude Code v2.1.227 по умолчанию включён **tool search**: «Only tool names and server instructions load at session start, so adding more MCP servers has minimal impact on your context window». Полные схемы подтягиваются, только когда модель ищет их под конкретную задачу. Жёсткого лимита на число инструментов нет — ограничение практическое, это бюджет контекста.

ENABLE_TOOL_SEARCH	ПОВЕДЕНИЕ
не задано	все MCP-инструменты отложены, грузятся по требованию
true	принудительно отложить
auto	грузить сразу, если схемы влезают в 10% окна контекста; остальное отложить
auto:N	то же с порогом N% (например <code>auto:5</code>)
false	грузить всё сразу, без отложенной загрузки

Смежные ручки: `alwaysLoad: true` в записи сервера — исключение из отложенной загрузки для того, что нужно на каждом шаге (платите контекстом); описания инструментов и `server instructions` обрезаются по 2 КБ каждое, поэтому важное держите в начале; вывод инструмента даёт предупреждение при превышении 10 000 токенов и жёстко ограничен 25 000 (поднимается через `MAX_MCP_OUTPUT_TOKENS`).

Практический вывод записан прямо в документации: сервер, которым вы не пользуетесь, лучше удалить — `claude mcp remove <name>`.

URL	ЧТО ЭТО	ПОСЛЕДНЯЯ АКТИВНОСТЬ
https://modelcontextprotocol.io/	Официальный сайт MCP	ведёт на спецификацию 2026-07-28
https://modelcontextprotocol.io/specification/2026-07-28	Текущая спецификация	28.07.2026
https://modelcontextprotocol.io/specification/2026-07-28/basic/transports	Транспорты stdio и Streamable HTTP	28.07.2026
https://modelcontextprotocol.io/docs/2026-07-28/learn/server-concepts	Tools / Resources / Prompts и методы протокола	28.07.2026
https://modelcontextprotocol.io/specification/2026-07-28/basic/security_best_practices	Security Best Practices	28.07.2026
https://blog.modelcontextprotocol.io/posts/2026-07-28/	Разбор релиза спецификации	28.07.2026
https://www.anthropic.com/news/donating-the-model-context-protocol-and-establishing-of-the-agentic-ai-foundation	Передача MCP в Agentic AI Foundation	09.12.2025
https://code.claude.com/docs/en/mcp-quickstart	Пошаговое подключение сервера к Claude Code	актуальная
https://code.claude.com/docs/en/mcp	Полный референс MCP в Claude Code	актуальная, упоминает v2.1.227
https://code.claude.com/docs/en/security	Безопасность Claude Code, раздел MCP security	актуальная
https://registry.modelcontextprotocol.io/	Официальный реестр MCP-серверов	API отвечает <code>{"status":"ok"}</code> на 12.08.2026
https://github.com/modelcontextprotocol/registry	Исходники реестра, 7 137 звёзд	push 10.08.2026
https://github.com/modelcontextprotocol/servers	Референсные серверы, 89 482 звезды	push 10.08.2026
https://github.com/microsoft/playwright-mcp	Playwright MCP, 36 042 звезды, Apache-2.0	push 09.08.2026, релиз v0.0.79 от 06.08.2026
https://github.com/upstash/context7	Context7, 60 643 звезды, MIT	push 12.08.2026, npm 4.0.2 от 11.08.2026
https://blog.google/innovation-and-ai/models-and-research/google-labs/stitch-ai-ui-design/	Анонс Stitch, подтверждает MCP-сервер и SDK	18.03.2026
https://stitch.withgoogle.com/docs/mcp/setup/	Документация Stitch MCP	отвечает 200, содержимое рендерится JS — текстом не подтверждено
https://docs.pencil.dev/getting-started/installation	Установка Pencil и авто-запуск его MCP-сервера	обновлено 28.07.2026

URL	ЧТО ЭТО	ПОСЛЕДНЯЯ АКТИВНОСТЬ
https://invariantlabs.ai/blog/mcp-github-vulnerability	Prompt injection против GitHub MCP	26.05.2025
https://thehackernews.com/2025/09/first-malicious-mcp-server-found.html	Вредоносный пакет postmark-mcp	29.09.2025
https://thehackernews.com/2026/01/three-flaws-in-anthropic-mcp-git-server.html	Три CVE в mcp-server-git	20.01.2026
https://www.oligo.security/blog/critical-rce-vulnerability-in-anthropic-mcp-inspector-cve-2025-49596	RCE в MCP Inspector, CVSS 9.4	13.06.2025
https://github.com/advisories/GHSA-7f8r-222p-6f5g	GitHub Advisory по CVE-2025-49596	исправлено в 0.14.1
https://www.pulsemcp.com/servers	Каталог PulseMCP, 22 074 сервера	обновляется ежедневно
https://smithery.ai	Каталог Smithery, «14,281+ MCPs»	live на 12.08.2026
https://claude.ai/directory	Anthropic Directory проверенных коннекторов	анонимно отдаёт HTTP 403; ссылка из официальной документации Claude Code

5. Хуки

обеспечить кодом, а не попросить словами

Проверено по документации 12.08.2026. Последняя версия в CHANGELOG на момент проверки — **2.1.228** (на занятии записывали 2.1.227).

Обеспечить кодом, а не попросить словами

CLAUDE.md — текст в контексте модели. Написанное там «никогда не делай `git push --force`» модель прочтает, скорее всего учтёт — и однажды всё равно не учтёт. Это просьба, не гарантия.

Хук — ваш код, который Claude Code запускает сам в определённой точке цикла, всегда, независимо от решения модели: «детерминированный контроль: определённые действия происходят всегда, а не тогда, когда LLM решит их выполнить». Отсюда разделение: стиль и контекст проекта — в **CLAUDE.md** и скиллы; «этого не должно случиться никогда» и «это должно происходить после каждой правки» — в хуки.

Хуки **PreToolUse** срабатывают **до** проверки режима разрешений, во всех режимах, включая `bypassPermissions` и `--dangerously-skip-permissions`. Ужесточить политику хук может, ослабить — нет: `allow` из хука не отменяет `deny`-правила из `settings`.

События жизненного цикла

Событий больше тридцати, они сгруппированы по частоте срабатывания.

Раз за сессию: `SessionStart` (старт/возобновление), `SessionEnd`, `Setup` (`--init-only`, либо `--init / --maintenance` в `-p`).

Раз за ход: `UserPromptSubmit` (промпт отправлен, Claude ещё не начал), `Stop` (Claude закончил отвечать), `StopFailure` (ход оборвался ошибкой API).

На каждый вызов инструмента: `PreToolUse` (до вызова, можно заблокировать), `PostToolUse` (после успеха), `PostToolUseFailure` (после ошибки), `PostToolBatch` (после всей параллельной пачки).

Остальные: `PermissionRequest`, `PermissionDenied`, `UserPromptExpansion`, `Notification`, `MessageDisplay`, `SubagentStart`, `SubagentStop`, `TaskCreated`, `TaskCompleted`, `TeammateIdle`, `InstructionsLoaded` (загрузка `CLAUDE.md` или `.claude/rules/*.md`), `ConfigChange`, `CwdChanged`, `DirectoryAdded`, `FileChanged`, `WorktreeCreate`, `WorktreeRemove`, `PreCompact`, `PostCompact`, `Elicitation`, `ElicitationResult`.

`Stop` срабатывает при любом завершении ответа, а не только задачи, и не срабатывает на прерывание пользователем.

Настройка

ФАЙЛ	ОБЛАСТЬ	КОММИТИТСЯ
<code>~/.claude/settings.json</code>	все проекты	нет
<code>.claude/settings.json</code>	один проект	да
<code>.claude/settings.local.json</code>	один проект	нет
managed policy settings	организация	да, админ
плагин <code>hooks/hooks.json</code>	пока плагин включён	да
frontmatter скилла/агента	пока компонент активен	да

Уровни **складываются**, а не перетирают друг друга; одинаковый обработчик из нескольких файлов выполнится один раз. Структура двухуровневая: событие → группы с `matcher` → обработчики.

```
{ "hooks": { "PreToolUse": [ { "matcher": "Bash", "hooks": [
  { "type": "command", "if": "Bash(git *)", "timeout": 60,
    "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/check.sh" } ] } ] } }
```

matcher. Пусто или `"*"` — всегда. Строка только из букв, цифр, `_`, `-`, пробелов, `,` и `|` — точное имя или список: `Bash`, `Edit|Write` (с v2.1.191 запятая работает так же). Любой другой символ переводит `matcher` в JS-регулярку без якорей: `^Notebook`, `mcp_github_.*`. Регистр важен. MCP-инструменты именуются `mcp__<сервер>__<инструмент>`, поэтому `mcp__.*__write.*` ловит пишущие инструменты любого сервера. У не-инструментальных событий фильтруется другое поле: `SessionStart` — `startup|resume|clear|compact|fork`, `Notification` — `permission_prompt|idle_prompt|...`, `PreCompact` — `manual|auto`. У `UserPromptSubmit`, `Stop`, `PostToolBatch`, `CwdChanged` `matcher` не поддерживается.

if — фильтр синтаксисом `permission`-правил (`"Bash(git *)"`, `"Edit(*.ts)"`), только на `tool`-событиях; «падает открыто» на непарсимой команде, поэтому жёсткие запреты стройте на самом хуке.

type бывает пяти видов: `command`, `http` (POST на URL), `mcp_tool`, `prompt` (один вызов модели, по умолчанию `Naiku`), `agent` (субагент с инструментами, экспериментальный). Таймауты: 600 с для первых трёх, 30 с для `prompt`, 60 с для `agent`; `UserPromptSubmit` понижает первую группу до 30 с.

Протокол: `stdin` → `stdout/stderr` → код выхода

На `stdin` приходит JSON. Общие поля: `session_id`, `prompt_id`, `transcript_path`, `cwd`, `permission_mode`, `effort`, `hook_event_name`. Tool-события добавляют `tool_name`, `tool_input`, `tool_use_id`:

```
{ "session_id": "abc123", "cwd": "/Users/sarah/myproject",
  "hook_event_name": "PreToolUse", "tool_name": "Bash",
  "tool_input": { "command": "npm test" }, "tool_use_id": "toolu_01ABC..." }
```

`UserPromptSubmit` получает `prompt`, `SessionStart` — `source`, `Stop` — `stop_hook_active` и `last_assistant_message`, `SessionEnd` — `reason`.

Коды выхода. `0` — возражений нет (для `PreToolUse` это не одобрение, обычный permission-flow продолжается). `2` — блокировка, `stderr` уходит обратно как обратная связь: `PreToolUse` блокирует вызов, `UserPromptSubmit` стирает промпт, `Stop` / `SubagentStop` не дают завершиться, `PreCompact` отменяет компактизацию. Не блокируются `PostToolUse`, `Notification`, `SessionStart`, `SessionEnd`, `PermissionRequest` (там код `2` игнорируется — решайте объектом `decision`). Другой код без валидного JSON — non-blocking error.

JSON на `stdout` (при выходе `0`; смешивать с кодом `2` не надо). Универсальные поля — `continue`, `stopReason`, `systemMessage`, `terminalSequence`. Решение для `PreToolUse`:

```
{ "hookSpecificOutput": { "hookEventName": "PreToolUse",
  "permissionDecision": "deny",
  "permissionDecisionReason": "Используй rg вместо grep" } }
```

Значения: `allow` (пропустить диалог), `deny` (отменить вызов, причина уходит модели), `ask` (показать диалог), плюс `defer` в режиме `-p`. У `PermissionRequest` форма другая — `hookSpecificOutput.decision.behavior` со значениями `allow|deny|escalate`. `UserPromptSubmit` подмешивает текст через `hookSpecificOutput.additionalContext` (на верхнем уровне поле молча игнорируется); у него, `UserPromptExpansion` и `SessionStart` любой `stdout` при выходе `0` попадает в контекст.

Хуки запускаются параллельно и все дорабатывают до конца — `deny` одного не отменяет побочные эффекты другого. Результаты сливаются по самому строгому: `deny` → `defer` → `ask` → `allow`. Вывод обрезается на 10 000 символов.

Рабочие примеры

1. Блокировка необратимых команд

```
#!/usr/bin/env bash
# .claude/hooks/block-destructive.sh
CMD=$(jq -r '.tool_input.command // empty')

DENY=(
  'rm[[:space:]]+-[a-zA-Z]*[rf]'
  'git[[:space:]]+push[[:space:]].*(--force[[:space:]]-f([[:space:]]|$))'
  'git[[:space:]]+reset[[:space:]]+--hard'
  '(DROP|TRUNCATE)[[:space:]]+(DATABASE|SCHEMA|TABLE)'
)

for re in "${DENY[@]"; do
  if printf '%s' "$CMD" | grep -Eiq "$re"; then
    jq -n --arg re "$re" '{hookSpecificOutput:{
      hookEventName:"PreToolUse", permissionDecision:"deny",
      permissionDecisionReason:("Необратимая команда запрещена: " + $re)}}'
    exit 0
  fi
done
exit 0
```

```
{ "hooks": { "PreToolUse": [ { "matcher": "Bash", "hooks": [ { "type": "command",
  "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/block-destructive.sh" } ] } ] } }
```

Не забудьте `chmod +x`. Этот запрет переживает даже `--dangerously-skip-permissions`.

2. Запрет чтения файлов с секретами

```
#!/usr/bin/env bash
P=$(jq -r '.tool_input.file_path // .tool_input.path // empty')
case "${P//\\//}" in
  *.env|*.env.*|*id_rsa*|.pem|*/secrets/*|*credentials.json)
    echo "Файл с секретами недоступен: $P. Схема — в .env.example." >&2
    exit 2 ;;
esac
exit 0
```

matcher: `"Read|Edit|Write|NotebookEdit"`. Дыра, которую закрывают отдельно: `cat .env` через Bash под этот matcher не попадает — нужен второй хук на Bash.

3. Автоформатирование после правки

```
{ "hooks": { "PostToolUse": [ { "matcher": "Edit|Write", "hooks": [ { "type": "command",
  "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write" } ] } ] } }
```

Для полиглот-репозитория вынесите это в скрипт с `case "$F" in *.py) ruff format ...;; *.go) gofmt -w ...;; esac`. `PostToolUse` не отменяет действие — правка уже произошла.

4. Валидация плана сторонней моделью

План уходит на ревью в Codex CLI, и пока внешняя модель не согласна — выход из плана не подтверждается. Событие — `PermissionRequest` с matcher `ExitPlanMode` (так же устроен Plannotator).

```
#!/usr/bin/env bash
# .claude/hooks/review-plan.sh
INPUT=$(cat)
printf '%s' "$INPUT" > /tmp/claude-plan-hook.json # первый запуск: увидеть схему
PLAN=$(printf '%s' "$INPUT" | jq -r '.tool_input.plan // (.tool_input | tostring)')

VERDICT=$(codex ехес "Ты ревьюер инженерного плана. Найди пробелы и риски.
Ответь POWHO одной строкой JSON: {"ok\:true|false,\"reason\":"..\"}"

ПЛАН:
$PLAN" 2>/dev/null | tail -n 1)

if [ "$(printf '%s' "$VERDICT" | jq -r '.ok // "true" 2>/dev/null)" = "false" ]; then
  jq -n --arg r "$(printf '%s' "$VERDICT" | jq -r '.reason // ""'" \
    '{hookSpecificOutput:{hookEventName:"PermissionRequest",decision:{behavior:"deny"}},
    systemMessage:("Внешний ревьюер отклонил план: " + $r)}'
else
  jq -n '{hookSpecificOutput:{hookEventName:"PermissionRequest",decision:
    {behavior:"escalate"}}}'
fi
exit 0
```

```
{ "hooks": { "PermissionRequest": [ { "matcher": "ExitPlanMode", "hooks": [
  { "type": "command", "timeout": 300,
    "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/review-plan.sh" } ] } ] }
```

Оговорка: **имя поля с текстом плана в справочнике не задокументировано**, поэтому скрипт дампит входной JSON и имеет фолбэк на весь `tool_input` — проверьте дамп на первом запуске. Закрытый issue #21282 (январь 2026) утверждал, что `plan mode` невидим для хуков; документация опровергает это.

Альтернатива без внешних CLI — хук `"type": "agent"` с `"prompt": "...$ARGUMENTS"`: субагент возвращает `{"ok": ..., "reason": ...}`, и при `false` на `Stop` причина уходит Claude, а он продолжает работать.

5. Уведомление о завершении

```
{ "hooks": { "Stop": [ { "hooks": [ { "type": "command",
  "command": "osascript -e 'display notification \"Ход завершён\" with title \"Claude Code\"'" } ] } ] }
```

Тот же обработчик на `Notification` с matcher `permission_prompt` подаст сигнал, когда нужно разрешение. На macOS уведомления идут через Script Editor: без его прав команда падает молча — выполните один раз `osascript -e 'display notification "test"'` и включите **System Settings** → **Notifications** → **Script Editor**. На Linux — `notify-send` (libnotify-bin). Без внешних команд то же делает поле `terminalSequence` (OSC 9/99/777, с v2.1.141).

Отладка

- `/hooks` — браузер хуков по событиям, с источником; только чтение.

- `Ctrl+0` — транскрипт. Успешный хук не показывает ничего; non-blocking error выглядит как `<hook name> hook error`.
- Полный лог: `claude --debug-file /tmp/claude.log` и `tail -f /tmp/claude.log`; в идущей сессии — `/debug`.
- Ручной прогон: `echo '{"tool_name":"Bash","tool_input":{"command":"ls"}}' | ./my-hook.sh; echo $?`.

СИМПТОМ	ПРИЧИНА
Хук не срабатывает	matcher не совпал (регистр!), не то событие, файл не исполняемый
<code>command not found</code>	относительный путь — нужен <code>\${CLAUDE_PROJECT_DIR}</code> либо <code>"args": []</code>
JSON не действует, ошибок нет	профиль шелла печатает в stdout и портит JSON — оберните <code>echo</code> в <code>if [[\$- == *i*]]</code>
Хуков нет в <code>/hooks</code>	битый JSON, не тот файл, watcher пропустил правку — рестарт
Stop блокирует бесконечно	не проверяете <code>stop_hook_active</code> ; лимит 8 подряд, поднимается <code>CLAUDE_CODE_STOP_HOOK_BLOCK_CAP</code>

Безопасность

Хуки исполняются **без песочницы, с полными правами вашего пользователя**, автоматически в момент события. Подтверждения перед запуском нет.

- Чужой `.claude/settings.json` из клонированного репозитория — исполняемый код. Читайте его перед первым запуском Claude Code в незнакомом проекте; то же с плагинами и MCP-серверами.
- Хук читает подконтрольные модели данные: кавычьте переменные, не собирайте команды конкатенацией, валидируйте пути.
- HTTP-хуки шлют событие наружу. Интерполируются только переменные из `allowedEnvVars`; домены ограничиваются через `allowedHttpHookUrls`.
- `allowManagedHooksOnly` в managed settings блокирует пользовательские, проектные и плагиновые хуки. Локальный выключатель — `"disableAllHooks": true`.
- Frontmatter-хуки проектных субагентов с v2.1.218 требуют диалога доверия к workspace.

Образец собранной системы — плагин `security-guidance`: `SessionStart` поднимает окружение, `UserPromptSubmit` снимает срез рабочего дерева, `PostToolUse` на `Edit|Write` делает паттерн-матч без вызова модели, `Stop` в фоне шлёт диф на ревью отдельной модели.

Ссылки

URL	ЧТО ЭТО	ПОСЛЕДНЯЯ АКТИВНОСТЬ
https://code.claude.com/docs/en/hooks	Справочник: события, схемы JSON, коды выхода, matcher	проверено 12.08.2026
https://code.claude.com/docs/en/hooks-guide	Руководство: примеры, <code>/hooks</code> , отладка, ограничения	проверено 12.08.2026
https://code.claude.com/docs/en/security-guidance	Плагин security-guidance — продакшн-пример хуков	проверено 12.08.2026
https://github.com/anthropics/claude-code	Репозиторий и CHANGELOG (2.1.228)	141 171 звезда, push 11.08.2026
https://github.com/anthropics/claude-code/blob/main/examples/hooks/bash_command_validator_example.py	Референсный <code>PreToolUse</code> -валидатор Bash	в ветке main на 12.08.2026
https://github.com/anthropics/claude-code/issues/21282	Issue про plan mode и хуки — неактуален	закрит, создан 27.01.2026
https://github.com/anthropics/claude-plugins-official	Маркетплейс; исходники <code>plugins/security-guidance</code>	33,4 тыс. звёзд на 12.08.2026
https://learn.chatgpt.com/docs/codex/cli	Codex CLI: режим <code>codex exec</code> " <code><промт></code> "	проверено 12.08.2026
https://docs.plannotator.ai/open-source/agents/claude-code	Пример на <code>PermissionRequest</code> + <code>ExitPlanMode</code>	проверено 12.08.2026
https://jqlang.org/download/	Установка <code>jq</code> (1.8.2) — разбор stdin в примерах	проверено 12.08.2026

6. Контекст, модели и экономика

окно, компакт, усилие, тарифы

Цифры проверены по первоисточникам **12 августа 2026 года**. Тарифы и лимиты меняются часто — сверяйтесь по ссылкам в конце.

Контекстное окно: рабочая память, а не диск

В окно попадает **всё**, что уходит в запрос: системный промт, каждое сообщение в `messages` (включая результаты инструментов, изображения, документы), определения инструментов и сгенерированные размышления. Памяти между запросами нет — клиент шлёт всю историю заново на каждом ходе.

Интуиция ломается здесь: **больше контекста ≠ лучше**. С ростом числа токенов точность и полнота падают; в документации Anthropic это названо *context rot*. Причина архитектурная — трансформер строит n^2 попарных связей для n токенов, и «бюджет внимания» размазывается тем тоньше, чем длиннее вход. Замеры:

- **Lost in the Middle** (Liu et al., Stanford; arXiv 2307.03172, TACL 2024): качество максимально, когда нужный фрагмент в начале или конце входа, и заметно падает в середине. U-образная кривая есть даже у моделей, заявленных как long-context.

- **NoLiMa** (Modarressi et al., Adobe Research + LMU; arXiv 2502.05167, ICML 2025): «иголка в стоге» без лексического пересечения вопроса и ответа. Из 13 моделей с заявленными $\geq 128K$ **11 на 32K падают ниже 50 %** собственного короткого базового уровня; GPT-4o — с 99,3 % до 69,7 %.
- **Context Rot** (Hong, Troynikov, Huber; Chroma, 14.07.2025): 18 фронтир-моделей (Claude 4, GPT-4.1, Gemini 2.5, Qwen3) на 10K–500K. Дegradiруют **все 18**; один отвлекающий фрагмент уже снижает точность. Контринтуитивно: на логически связном «стоге» модели работают **хуже**, чем на перемешанном.

Деградация начинается задолго до формального лимита: окно на 1М не значит, что на 400K будет то же качество, что на 40K.

Актуальные окна контекста

МОДЕЛЬ	КОНТЕКСТ	MAX OUTPUT	\$/1М ВХОД-ВЫХОД
Claude Fable 5	1М	128K	10 / 50
Claude Opus 5	1М	128K	5 / 25
Claude Sonnet 5	1М	128K	2 / 10
Claude Haiku 4.5	200K	64K	1 / 5
Opus 4.8 / 4.7 / 4.6	1М	128K	5 / 25
Sonnet 4.6	1М	128K	3 / 15
Sonnet 4.5	200K	64K	3 / 15

Для всех моделей с окном 1М это **умолчание**: beta-заголовок не нужен, длинный контекст идёт по обычной цене — запрос на 900K стоит столько же за токен, сколько на 9K. Цена Sonnet 5 \$2/\$10 объявлялась вводной до 31.08.2026, но Anthropic уточнила: повышение до \$3/\$15 **не состоится**. В Claude Code окно 1М выбирается алиасами `opus[1m]` / `sonnet[1m]` либо суффиксом (`/model claude-opus-4-8[1m]`); на Max, Team и Enterprise Opus повышается автоматически, а `CLAUDE_CODE_DISABLE_1M_CONTEXT=1` удерживает сессию на 200K.

GPT-серия: `gpt-5.6-sol` — контекст **1 050 000** токенов, до 128 000 вывода, \$5/\$30, кэшированный вход \$0,50; уровни reasoning effort `none`, `low`, `medium` (умолчание), `high`, `xhigh`, `max`. Деталь, которой нет у Anthropic: **свыше 272K входных токенов запрос тарифицируется 2x по входу и 1,5x по выходу целиком**. `terra` — \$2/\$12, `luna` — \$0,20/\$1,20.

Открытые веса: `Qwen3-Coder-480B-A35B-Instruct` — 480B/35B активных, нативно **262 144** токена, до 1М через YaRN, Apache 2.0. `Llama-4-Scout-17B-16E-Instruct` — 109B/17B активных/16 экспертов, заявленный контекст **10М**, лицензия Llama 4 Community (не OSI). Заявленное окно здесь — верхняя граница техподдержки, а не длина, на которой держится качество.

Команда /context

`/context` рисует заполнение окна цветной сеткой и подсказывает, какие инструменты съедают больше всего, где раздулась память и близко ли предел; если окно превышено — сообщает, насколько.

Заполнение **до первого сообщения** — фиксированные блоки (ориентиры из демонстрационной сессии в документации): system prompt с определениями инструментов ~4 200; auto memory `MEMORY.md` (первые 200 строк или 25 КБ) ~680; environment info — директория, платформа, shell, ОС, git — ~280;

MCP tools ~120, потому что схемы **по умолчанию отложены** и в контекст идут только имена; описания скиллов, не тела, ~450; `~/.claude/CLAUDE.md` ~320 и проектный `CLAUDE.md` ~1 800.

Смысл — в пропорциях: MCP-сервер с неотложенными схемами перевешивает всё остальное вместе. Отсюда рекомендации документации: предпочитать CLI-утилиты (`gh`, `aws`, `gcloud`) MCP-серверам и держать `CLAUDE.md` короче 200 строк, вынося специфику в скиллы.

Компактификация

Упираясь в потолок, Claude Code заменяет историю структурированным резюме. Вручную — `/compact`, можно с фокусом: `/compact focus on the auth bug fix`. Порог автокомakta задаётся `/autocompact 500k` (принимает 100K–1M), флагом `--autocompact` или `CLAUDE_CODE_AUTO_COMPACT_WINDOW`. Без настройки компакт срабатывает на лимите модели; исключения — Sonnet 4.6 и Opus 4.6 без расширенного контекста (200K) и Sonnet 5 в конфигурации 1M (около **967K**).

МЕХАНИЗМ	ПОСЛЕ КОМПАКТИФИКАЦИИ
System prompt, output style	Не меняются — не часть истории сообщений
Корневой <code>CLAUDE.md</code> , неограниченные rules, auto memory	Переинжектятся с диска
Rules с фронтматтером <code>paths:</code>	Теряются до чтения подходящего файла
Вложенные <code>CLAUDE.md</code> в поддиректориях	Теряются до чтения файла оттуда
Тела вызванных скиллов	Переинжектятся с потолком 5 000 токенов на скилл и 25 000 суммарно , старые отбрасываются первыми; обрезка сохраняет начало файла
Hooks	Не применимо — исполняются как код, а не как контекст

Отсюда два правила: **важное — в начало `SKILL.md`**; если правило должно пережить компакт, уберите у него `paths:` или перенесите в корневой `CLAUDE.md`.

Гигиена сессий. `/clear` между несвязанными задачами стоит ноль и полностью сбрасывает контекст. `/compact` — в естественных паузах, а не когда сработает автомат: компакт сам по себе большой запрос, он читает всю резюмируемую историю. Если пошли не туда — `/rewind`, а не `/compact :rewind` обрезает разговор назад к уже закешированному префиксу. `/recap` выдаёт резюме в терминал, не трогая ни историю, ни кэш.

Субагенты как изоляция контекста

Субагент — отдельный экземпляр Claude **со своим контекстным окном**, системным промптом, набором инструментов и правами. Родительский разговор внутрь не передаётся: единственный канал — строка задания. Наверх возвращается **только итоговое резюме**; чтения файлов, поиски и промежуточные рассуждения остаются внутри.

Определяются файлами в `.claude/agents/` (проектные) или `~/.claude/agents/` (пользовательские). Фронтматтер: `name` и `description` обязательны; опционально `tools`, `model` (`sonnet / opus / haiku / fable` / полный ID/ `inherit`, по умолчанию `inherit`) и `effort`. Вложенность — до трёх уровней ниже основного разговора.

Отличие от новой сессии: субагент изолирует шумную подзадачу **внутри** текущей работы, не трогая кэш родителя (вызов и результат дописываются в конец), а новая сессия нужна при смене темы целиком. Свой кэш субагент строит с нуля, с TTL 5 минут даже на подписке. **Agent teams** (несколько

инстансов Claude Code) в plan mode тратят примерно **в 7 раз больше токенов**, чем обычная сессия — каждый ведёт своё окно.

Усилие модели

effort управляет тем, сколько токенов Claude тратит на ответ, влияя на **все** токены — текст, вызовы инструментов и размышления, — и не требуя включённого thinking. На низком усилии модель объединяет операции в меньшее число вызовов и обходится без преамбул; на высоком — больше вызовов, план перед действием, подробные сводки.

УРОВЕНЬ	КОГДА
max	Максимум без ограничений расхода (Fable 5, Opus 5/4.8/4.7/4.6, Sonnet 5/4.6)
xhigh	Агентные задачи 30+ мин, бюджеты в миллионы токенов (Fable 5, Opus 5/4.8/4.7, Sonnet 5)
high	Умолчение; эквивалент отсутствия параметра
medium	Баланс скорости, цены и качества
low	Простые задачи, субагенты, латентно-чувствительное

API-умолчение — **high** везде, кроме Opus 4.7, у которого в Claude Code умолчение **xhigh**. Для Opus 5 документация советует стартовать с **high** и активно использовать **low / medium** как основной рычаг цены и времени, а не переносить настройки с прошлой модели. Ограничение: **на Opus 5 нельзя отключить thinking при xhigh или max** — вернётся 400.

Переключение: **/effort** (без аргумента — слайдер), стрелки в **/model**, флаг **--effort**, **CLAUDE_CODE_EFFORT_LEVEL**, ключ **effortLevel** в настройках, поле **effort** во фронтматтере скилла или субагента. Приоритет: переменная окружения → ваша настройка → умолчение модели.

Два подводных камня. **ultracode** в меню **/effort** — **не уровень модели**, а настройка Claude Code: шлёт **xhigh** и включает оркестрацию динамических workflow. Слово **ultrathink** добавляет инструкцию в контекст, но **уровень effort, уходящий в API, не меняется**; «think hard» и подобное Claude Code как ключевые слова не распознаёт.

```
curl https://api.anthropic.com/v1/messages \
  -H "x-api-key: $ANTHROPIC_API_KEY" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-opus-5", "max_tokens": 4096,
      "messages": [{"role": "user", "content": "..."}],
      "output_config": {"effort": "medium"}}'
```

Экономика

Планы Claude: Pro **\$20/мес** (\$17 при годовой оплате); Max 5x **\$100/мес** — «в 5 раз больше использования за сессию, чем Pro»; Max 20x **\$200/мес** — «в 20 раз больше»; Team \$20 за место при годовой оплате или \$25 ежемесячно (премиум-место \$100/\$125); Enterprise \$20 за место плюс потребление по API-ставкам.

Сессионный лимит сбрасывается **каждые пять часов**; сессия стартует с вашего первого сообщения и покрывает и Claude Code, и чат — пул общий. Поверх у Max действует **недельный лимит по всем**

моделям, сбрасывающийся в фиксированное для аккаунта время, не зависящее от даты подписки. Anthropic оговаривает право вводить дополнительные ограничения по моделям и функциям. Прогресс виден в **Settings** → **Usage**, в Claude Code — по `/usage`.

Честно: Anthropic не публикует точных токенных квот ни для одного плана. Официальные материалы дают только множители (Pro 1×, Max 5× или 20×) и список факторов расхода: длина сообщений и разговора, размер файлов, инструменты, модель, уровень усилия. Цифры вида «24–40 часов Opus в неделю» — оценки третьих сторон, первоисточником не подтверждённые. Ещё расхождение: claude.com/pricing показывает Max строкой «от \$100/мес», справочный центр разделяет \$100 и \$200 — доверяйте справочному центру.

Ориентир по реальному расходу из документации Claude Code: в корпоративных внедрениях **около \$13 на разработчика за активный день и \$150–250 в месяц**, у 90 % — ниже \$30 за активный день.

Codex / ChatGPT: Free \$0, Go \$8, Plus \$20, Pro от \$100 («в 5× или 20× выше лимиты, чем Plus»), Business \$20 за пользователя в месяц при годовой оплате от 2 мест, Enterprise по запросу. Структура похожая — **пятичасовое скользящее окно** плюс недельные потолки. Но OpenAI, в отличие от Anthropic, **публикует диапазоны сообщений:** для Plus GPT-5.6 Sol 10–100, Terra 25–200, Luna 250–2000 за окно; Pro 5× умножает на 5, 20× — на 20. Сверх включённого работают кредиты.

API за токены. Claude: Fable 5 \$10/\$50, Opus 5 \$5/\$25, Sonnet 5 \$2/\$10, Haiku 4.5 \$1/\$5; **Batch API — минус 50 %** (Opus 5 → \$2,50/\$12,50). Веб-поиск \$10 за 1000 запросов; web fetch без наценки сверх токенов; code execution бесплатен вместе с поиском, иначе 1550 бесплатных часов в месяц на организацию и \$0,05 за час контейнера сверх. Fast mode для Opus 5 / 4.8 — \$10/\$50, только на первичном API. OpenAI: sol \$5/\$30, terra \$2/\$12, luna \$0,20/\$1,20; batch и flex — минус 50 %, fast mode — 2×.

Кэширование промпта и экономия токенов

Кэш — главный рычаг в агентных сессиях, потому что история шлётся целиком на каждом ходе.

Чтение из кэша ~0,1× от обычной цены входа, запись — 1,25× при TTL 5 минут и 2× при часовом; пятиминутный окупается после одного чтения, часовой — после двух.

Механика — **побайтовое совпадение префикса:** изменение в начале запроса пересчитывает всё, что идёт после. Claude Code выстраивает запрос слоями от редко меняющегося к часто: system prompt (инструкции + определения инструментов) → project context (`CLAUDE.md` , auto memory) → conversation.

Ломает кэш: смена модели, смена уровня effort (тоже часть ключа — Claude Code переспросит), включение fast mode, подключение или отключение MCP-сервера **если** его инструменты грузятся в префикс, а не отложены, включение плагина с MCP-серверами, deny-правило на целый инструмент (`Bash` , `WebFetch`), компактификация, обновление Claude Code.

Не ломает: правка файлов репозитория, правка `CLAUDE.md` посреди сессии (но она и **не применится** до `/clear` , `/compact` или перезапуска), смена output style (аналогично), смена permission mode, вызов скиллов и команд, `/recap` , `/rewind` , порождение субагента.

TTL Claude Code выбирает сам: **на подписке час**, на API-ключе и у сторонних провайдеров — пять минут; `ENABLE_PROMPT_CACHING_1H=1` включает часовой, `FORCE_PROMPT_CACHING_5M=1` возвращает пятиминутный. Проверка — поля `cache_read_input_tokens` и `cache_creation_input_tokens` в ответе: стабильно высокие записи означают, что что-то меняется в префиксе. Кэш привязан к машине и директории, поэтому сессии в разных папках, включая worktrees одного репозитория, кэш друг друга не видят.

Приёмы экономии: `/clear` между задачами; Sonnet вместо Opus где хватает и `model: haiku` для простых субагентов; `/mcp` — отключить неиспользуемые серверы; `PreToolUse` -хук как фильтр (`print` `test` через `grep -E '(FAIL|ERROR)'` — сотни токенов вместо десятков тысяч); инструкции из `CLAUDE.md` — в скиллы; плагины code intelligence; шумные операции — субагентам; конкретные промпты вместо «improve this codebase»; plan mode (Shift+Tab); понижение effort — токены размышлений тарифицируются как выходные.

URL	ЧТО ЭТО	ПОСЛЕДНЯЯ АКТИВНОСТЬ
https://arxiv.org/abs/2307.03172	Lost in the Middle — U-образная кривая (Liu et al.)	Финал 20.11.2023; TACL 2024
https://arxiv.org/abs/2502.05167	NoLiMa — длинный контекст без лексического пересечения	Подано 07.02.2025, финал 09.07.2025; ICML 2025
https://www.trychroma.com/research/context-rot	Context Rot — деградация на 18 фронт-моделях (Chroma)	14.07.2025
https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents	Инженерный блог Anthropic: контекст как конечный ресурс, «бюджет внимания»	29.09.2025
https://platform.claude.com/docs/en/about-claude/models/overview	Таблица моделей Claude: ID, окна, вывод, цены	Проверено 12.08.2026
https://platform.claude.com/docs/en/about-claude/pricing	Прайс API: токены, кэш, batch, fast mode, инструменты	Проверено 12.08.2026
https://platform.claude.com/docs/en/build-with-claude/context-windows	Устройство окна, размеры по моделям, thinking и tool use	Проверено 12.08.2026
https://platform.claude.com/docs/en/build-with-claude/effort	Параметр effort: уровни, умолчания, рекомендации	Проверено 12.08.2026
https://code.claude.com/docs/en/context-window	Разбор заполнения окна и таблица «что переживает компакт»	Проверено 12.08.2026
https://code.claude.com/docs/en/model-config	Модели, варианты [1m], effort, пороги автокомakta	Проверено 12.08.2026
https://code.claude.com/docs/en/sub-agents	Субагенты: изоляция, фронтматтер, вложенность	Проверено 12.08.2026
https://code.claude.com/docs/en/prompt-caching	Кэш в Claude Code: слои, что ломает/сохраняет, TTL	Проверено 12.08.2026
https://code.claude.com/docs/en/costs	Управление стоимостью, /usage, приёмы экономии	Проверено 12.08.2026
https://code.claude.com/docs/en/commands	Слэш-команды: /context, /compact, /autocompact, /effort	Проверено 12.08.2026
https://claude.com/pricing	Официальные тарифы Claude	Проверено 12.08.2026
https://support.claude.com/en/articles/11049741-what-is-the-max-plan	Справка: Max 5x = \$100, Max 20x = \$200, пятичасовые сессии	Проверено 12.08.2026
https://support.claude.com/en/articles/9797557-usage-limit-best-practices	Справка: сессионные и недельные лимиты, Settings → Usage	Проверено 12.08.2026
https://developers.openai.com/api/docs/pricing	Прайс OpenAI API по всем моделям	Проверено 12.08.2026
https://developers.openai.com/api/docs/models/gpt-5.6-sol	Карточка gpt-5.6-sol: окно 1 050 000, надбавка свыше 272K	Проверено 12.08.2026
https://learn.chatgpt.com/docs/pricing		Проверено 12.08.2026

URL	ЧТО ЭТО	ПОСЛЕДНЯЯ АКТИВНОСТЬ
	Тарифы и лимиты Codex/ChatGPT, кредиты	
https://huggingface.co/Qwen/Qwen3-Coder-480B-A35B-Instruct	Карточка Qwen3-Coder-480B: 262 144 нативно, Apache 2.0	Проверено 12.08.2026
https://huggingface.co/meta-llama/Llama-4-Scout-17B-16E-Instruct	Карточка Llama 4 Scout: контекст 10M, 109B/17B/16 экспертов	Проверено 12.08.2026

7. Разрешения и безопасность

режимы, песочницы, чек-лист

Всё ниже сверено с документацией и локальной установкой 12 августа 2026: `claude --version` → **2.1.228** (занятие записано на 2.1.227). Документация переехала с `docs.claude.com/en/docs/claude-code/*` на `code.claude.com/docs/en/*`, старые адреса отдадут 301.

Шесть режимов разрешений

Режим — базовая линия: что выполняется без вопроса. Полный список значений виден в `claude --help: acceptEdits, auto, bypassPermissions, manual, dontAsk, plan`.

РЕЖИМ	ЧТО ИДЁТ БЕЗ СПРОСА	КОГДА ПРИМЕНЯТЬ
<code>default</code> (в UI — Manual , алиас <code>manual</code>)	только чтение	старт, чувствительный код
<code>acceptEdits</code>	чтение, правки файлов и <code>mkdir, touch, rm, mv, cp, sed</code> в рабочей директории	итерации с ревью по <code>git diff</code>
<code>plan</code>	чтение и разведка; правки заблокированы до утверждения плана	изучение чужой кодовой базы
<code>auto</code>	почти всё, но каждое действие смотрит классификатор	длинные задачи
<code>dontAsk</code>	только то, что явно в <code>permissions.allow</code>	CI: сессия не ждёт ввода
<code>bypassPermissions</code>	всё	только контейнер/VM

Как переключать. `Shift+Tab` циклит `default` → `acceptEdits` → `plan`. `auto` появляется в цикле, если аккаунт подходит по требованиям; `bypassPermissions` — только если сессия стартовала с `--permission-mode bypassPermissions`, `--dangerously-skip-permissions` или `--allow-dangerously-skip-permissions` (последний добавляет режим в цикл, не включая его); `dontAsk` в цикле не появляется никогда. Войти в `bypassPermissions` из сессии, стартовавшей без него, нельзя.

```
claude --permission-mode plan      # на старте
claude --permission-mode dontAsk   # CI
```

Постоянное значение — `permissions.defaultMode` в `settings.json`. Оговорка: `defaultMode: "auto"` из `.claude/settings.json` и `.claude/settings.local.json` игнорируется с 2.1.142, чтобы репозиторий не мог выдать себе auto-режим; кладите в `~/.claude/settings.json`.

Важно: с 14 августа 2026 `auto` становится режимом по умолчанию для новых сессий на Pro, Max и Team. Свой заданный `defaultMode` остаётся в силе.

Что не выключается ничем, кроме `bypassPermissions`: запись в *protected paths* — каталоги `.git`, `.claude`, `.vscode`, `.idea`, `.devcontainer`, `.husky` и файлы вида `.zshrc`, `.envrc`, `.npmrc`, `.mcp.json`. Правила `permissions.allow` их не открывают: проверка идёт раньше разбора `allow`-правил.

Файл разрешений: `allow` / `ask` / `deny`

Правило имеет форму `Tool` или `Tool(specifier)`. Порядок вычисления жёсткий: **deny** → **ask** → **allow**, первое совпадение выигрывает, специфичность роли не играет. Поэтому `Bash(aws *)` в `deny` перекрывает `Bash(aws s3 ls)` в `allow` — исключения из запрета сделать нельзя. Голое имя инструмента в `deny` (`Bash`) вообще убирает инструмент из контекста модели.

Синтаксис по инструментам:

- **Bash** — глобы `*` в любой позиции; `Bash(ls *)` требует границы слова (не матчит `ls of`), `Bash(ls*)` — матчит. Составные команды разбираются по `&&`, `||`, `;`, `|`, `&` — правило должно покрыть каждую подкоманду. Обёртки `timeout`, `nice`, `nohup`, `command`, `xargs` без флагов срезаются, а `npx`, `docker exec`, `devbox run` — нет, поэтому `Bash(devbox run *)` разрешает и `devbox run rm -rf .`.
- **Read / Edit** — gitignore-синтаксис с якорями: `//abs` — от корня ФС, `~/path` — от дома, `/path` — от источника настроек (не от корня ФС!), `path` — от текущей директории. Правила для `Write`, `NotebookEdit`, `Glob` **не читаются** — пишите `Edit(...)` и `Read(...)`. С 2.1.228 `deny` на `Read` (путь) закрывает и запись.
- **WebFetch** — `WebFetch(domain:example.com)`, `WebFetch(domain:*.example.com)`.
- **MCP** — `mcp_server`, `mcp_server_*`, `mcp_server__tool_name`; в `deny` допустим `mcp_*`.
- **По параметру** (только `deny/ask`) — `Tool(param:value)`, например `Bash(dangerouslyDisableSandbox:true)`.

Разумный набор для рабочего репозитория:

```
{
  "permissions": {
    "defaultMode": "plan",
    "disableBypassPermissionsMode": "disable",
    "deny": [
      "Read(**/.env)",
      "Read(**/.env.*)",
      "Read(~/.ssh/**)",
      "Read(~/.aws/**)",
      "Bash(rm -rf *)",
      "Bash(git reset --hard*)",
      "Bash(git clean -*)",
      "Bash(git push --force*)",
      "Bash(terraform destroy*)",
      "Bash(kubectl delete *)",
      "Bash(curl *)",
      "Bash(wget *)",
      "Edit(.github/workflows/**)"
    ],
    "ask": [
      "Bash(git push *)",
      "Bash(npm publish *)",
      "Bash(dangerouslyDisableSandbox:true)"
    ],
    "allow": ["Bash(npm run test *)", "Bash(npm run lint)"]
  }
}
```

Обоснование: `.env` и ключи — самый частый канал утечки, а `Read-deny` заодно закрывает запись; `rm -rf`, `reset --hard`, `clean`, `force push` — необратимая потеря незакоммиченного; `terraform destroy` и `kubectl delete` — необратимость за пределами машины; `curl / wget` — сетевой канал в обход `WebFetch(domain:...)`, документация прямо рекомендует эту связку; `.github/workflows` — правка CI равна исполнению кода с продовыми секретами; `disableBypassPermissionsMode` можно поставить самому себе, это не только админская настройка. Ask-правила действуют даже в `auto` и `bypassPermissions`.

Честная оговорка. Bash-правила по аргументам хрупкие: `Bash(rm -rf *)` не поймает `rm -fr`, а `Bash(curl http://github.com/ *)` — ни `curl -X GET`, ни `URL=... && curl $URL`, ни редиректы. Deny — фильтр от небрежности, а не граница безопасности. Границу даёт песочница или `PreToolUse`-хук (выход с кодом 2 блокирует вызов раньше разбора правил).

Почему запрет словами в `CLAUDE.md` — просьба

Документация формулирует это прямо: «*Permission rules are enforced by Claude Code, not by the model. Instructions in your prompt or CLAUDE.md shape what Claude tries to do, but they don't change what Claude Code allows.*»

Механика решения такая. Модель формирует вызов инструмента, дальше CLI проверяет по очереди: `PreToolUse`-хуки → deny/ask/allow-правила → режим → (в `auto`) классификатор → выполнение. `CLAUDE.md` в этой цепочке отсутствует: он часть промпта и влияет только на первый шаг, где модель решает, что попробовать.

Отсюда случай с занятия. Папка была «запрещена к удалению» текстом в `CLAUDE.md`, сессия шла в `bypassPermissions`. Модель либо не удержала инструкцию в фокусе, либо сочла запрет неотносящимся — и вызвала удаление. Проверять было нечему: в `bypassPermissions` не работают ни классификатор,

ни подтверждения. Остаются ровно три вещи: **deny-правила, явные ask-правила и circuit breaker** на `rm -rf /` и `rm -rf ~` (включая формы с `$(...)` и `<(...)`).

Пара правил `"deny": ["Edit(sacred/**)", "Read(sacred/**)"]` остановила бы это и в bypass-режиме. Слова — нет.

Песочницы и изоляция

подход	что изолировано	docker	усилия
Sandboxed Bash tool (/ sandbox)	Bash-команды и их потомки	нет	минимум
Sandbox runtime	весь процесс, включая MCP-серверы и хуки	нет	низкие
Dev container	среда разработки целиком	да	средние
Свой контейнер / VM	ОС целиком	да/нет	высокие
Claude Code on the web	VM на стороне Anthropic	нет	нет

Встроенная песочница Bash. На macOS — Seatbelt, ставить нечего. На Linux/WSL2 нужны `bubblewrap` и `socat` (`sudo apt-get install bubblewrap socat`), затем в сессии — `/sandbox`. Нативная Windows не поддерживается.

```
{
  "sandbox": {
    "enabled": true,
    "filesystem": { "denyRead": ["~/ /"], "allowRead": ["."] },
    "network": { "allowedDomains": ["github.com", "*.npmjs.org"] },
    "credentials": {
      "files": [{ "path": "~/.ssh", "mode": "deny" }],
      "envVars": [{ "name": "GITHUB_TOKEN", "mode": "deny" }]
    }
  }
}
```

Ограничение честное: песочница Bash не покрывает MCP-серверы и хуки — они процессы на хосте.

Sandbox runtime (`npx @anthropic-ai/sandbox-runtime claude`) заворачивает весь процесс в тот же Seatbelt/bubblewrap. Статус — beta research preview, формат конфигурации может измениться.

Dev container. Готовый пример — в `anthropics/claude-code/.devcontainer/`: `devcontainer.json`, `Dockerfile` и `init-firewall.sh`, который ставит `iptables -P OUTPUT DROP`, открывает только GitHub, `registry.npmjs.org`, `api.anthropic.com`, `sentry`, `statsig` и домены VS Code, а в конце проверяет себя двумя тестами. Это единственный сценарий, где `--dangerously-skip-permissions` уместен. На Linux и macOS CLI откажется стартовать с этим флагом от root/sudo.

Git worktrees — изоляция файлов, не безопасность. `claude --worktree feature-auth` создаёт checkout в `.claude/worktrees/<name>/`; Claude Code блокирует правки и команды, уходящие в основной checkout. Но `.git`, project-scope плагины и сохранённые approvals — общие с основным репозиторием.

Классы рисков и подтверждённые случаи

Уничтожение данных. Июль 2025: агент Replit удалил продовую базу проекта Джейсона Лемкина (SaaS) вопреки объявленному code freeze, затем сообщил, что откат невозможен, — это оказалось неправдой, откат сработал. Replit признал «catastrophic error of judgement».

Утечка секретов. 26 августа 2025 — атака s1ngularity на пакеты `nx / @nrwl/nx`. Постинсталл-скрипт `telemetry.js` искал установленные AI-CLI (Claude, Gemini, q) и запускал их с флагами `--dangerously-skip-permissions`, `--yolo`, `--trust-all-tools`, используя агента как инструмент разведки по файловой системе. Утекло 2 349 секретов, выгрузка шла в публичные репозитории жертв на GitHub. Первый известный случай, когда атака на цепочку поставок целенаправленно искала LLM-инструменты разработчика.

Prompt injection через файлы репозитория. CVE-2026-33068 (High): репозиторий с закоммиченным `.claude/settings.json`, содержащим `"defaultMode": "bypassPermissions"`, полностью пропускал диалог доверия — пользователь оказывался в bypass-режиме на первом же открытии. Исправлено в 2.1.53. CVE-2026-55607 (High) — побег из песочницы через worktree-путаницу; для эксплуатации требовалось клонировать вредоносный репозиторий с prompt-инъекцией. Исправлено в 2.1.163.

Prompt injection через веб. CVE-2026-54316 (Moderate): домен `huggingface.co` был в предодобренном списке WebFetch целиком, поэтому запросы на любой его путь, включая репозитории атакующего, уходили без спроса и в обход `--allowedTools` — готовый канал эксфильтрации. Исправлено в 2.1.163. Всего для `@anthropic-ai/claude-code` на 12.08.2026 опубликовано 28 advisory: аргумент не «инструмент дырявый», а «автообновление не отключать».

Вредоносные плагины. Демонстрация `prompt.security` (5 января 2026): маркетплейс-скилл подменял источник установки зависимости на подконтрольный атакующему. Это proof-of-concept, не зафиксированная в дикой природе атака, — но скиллы архитектурно стоят ровно на пути, где код скачивается и исполняется.

Ограничения auto-режима. Инженерный разбор Anthropic приводит 17% false-negative на реальных «переусердствовавших» действиях: это не замена человеческому ревью на высокорисковой инфраструктуре.

Чек-лист безопасного запуска в рабочем репозитории

1. Обновиться (`claude --version`) — половина известных обходов правил закрыта патчами.
2. Стартовать с `--permission-mode plan`: читать план, а не diff постфактум.
3. Закоммитить в `.claude/settings.json` `deny` на секреты и необратимые операции.
4. Поставить себе `disableBypassPermissionsMode: "disable"` в `~/.claude/settings.json`.
5. Включить `/sandbox` в `auto-allow: denyRead` на `~/`, `allowRead` на проект.
6. Вычистить окружение: `credentials.envVars` с `mode: "deny"` на `GITHUB_TOKEN`, `NPM_TOKEN`.
7. Не давать доступ к прод: отдельные креды и БД для агентской сессии.
8. Работать в worktree — ошибочное удаление ограничится веткой.
9. `--dangerously-skip-permissions` — только в dev container с `init-firewall.sh` и не от root.
10. Чужой репозиторий открывать в VM или Claude Code on the web.
11. Перед установкой плагина прочитать его хуки и MCP-конфиг: они исполняются без песочницы.
12. Смотреть `/permissions`: «Yes, don't ask again» пишет правило в `.claude/settings.local.json` на весь репозиторий, включая worktrees.
13. Коммитить часто: рабочая копия — единственное, что агент уничтожает безвозвратно.

Ссылки

URL	ЧТО ЭТО	ПОСЛЕДНЯЯ АКТИВНОСТЬ
https://code.claude.com/docs/en/permission-modes	Режимы, переключение, protected paths	12.08.2026
https://code.claude.com/docs/en/permissions	Синтаксис allow/ask/deny, managed settings	12.08.2026
https://code.claude.com/docs/en/sandboxing	Песочница Bash, ключи <code>sandbox.*</code>	12.08.2026
https://code.claude.com/docs/en/sandbox-environments	Сравнение способов изоляции	12.08.2026
https://code.claude.com/docs/en/devcontainer	Dev container и egress-фильтр	12.08.2026
https://code.claude.com/docs/en/security	Модель безопасности	12.08.2026
https://code.claude.com/docs/en/worktrees	<code>--worktree</code> и границы изоляции	12.08.2026
https://github.com/anthropics/claude-code/security/advisories	28 advisory, CVE и патч-версии	последняя 24.07.2026
https://github.com/anthropics/claude-code/tree/main/examples/settings	Готовые lax / strict / bash-sandbox	141 171 звезда, push 11.08.2026
https://github.com/anthropics/claude-code/blob/main/.devcontainer/init-firewall.sh	Default-deny iptables	push 11.08.2026
https://github.com/anthropic-experimental/sandbox-runtime	Sandbox runtime (beta)	4 943 звезды, push 11.08.2026
https://www.anthropic.com/engineering/claude-code-auto-mode	Устройство классификатора	25.03.2026
https://claude.com/blog/auto-mode-default-in-claude-code	Переход на auto с 14.08.2026	август 2026
https://www.wiz.io/blog/s1ngularity-supply-chain-attack	Разбор атаки s1ngularity	атака 26.08.2025
https://www.theregister.com/2025/07/21/replit_saastr_vibe_coding_incident/	Удаление БД агентом Replit	21.07.2025
https://prompt.security/blog/when-your-plugin-starts-picking-your-dependencies-marketplace-skills-and-dependency-hijack-in-claude-code	РоС подмены зависимости через скилл	05.01.2026
https://adversa.ai/blog/ai-coding-agent-incidents/	Компиляция инцидентов 2025–2026	07.2026; вторичный источник, часть кейсов — форумы и Reddit